

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ТЕРМИНОВ.....	3
ВВЕДЕНИЕ.....	4
1 Анализ предметной области.....	8
1.1 Машинное и глубокое обучение.....	8
1.2 Задача автономной навигации транспортного средства.....	9
1.3 Обзор подходов к решению задач.....	10
1.4 Метод условного имитационного обучения (CIL)	10
1.5 Рекуррентные нейронные сети в задачах управления	10
1.6 Моделирование в виртуальных средах	13
2 Постановка задачи и требования к системе	15
2.1 Цели и задачи разработки.....	15
2.2 Функциональные требования.....	15
2.3 Требования к данным и аппаратному обеспечению	16
3 Сравнение с аналогами	18
3.1 CILRS — Conditional Imitation Learning with Speed Prediction	18
3.2 Learning by Cheating (LBC) и Roach	18
3.3 TransFuser.....	18
3.4 Обоснование собственной разработки	18
3.5 Рекуррентные и гибридные модели в смежных задачах	20
4 Выбор платформы и инструментальных средств	21
4.1 Симулятор CARLA.....	21
4.2 Язык программирования и фреймворк.....	21
4.3 Предобученная CNN-архитектура ResNet-18	21
4.4 Лидарный датчик и BEV-представление	22
5 Описание разработки программного обеспечения	24
5.1 Общая архитектура модели CILDriveNet.....	24
5.2 Модуль обработки изображений с камеры	25

отформатировано: Шрифт: 14 пт

Отформатировано: междустрочный, 1,5 строки

5.3 Модуль обработки данных лидара (LidarEncoder)	25
5.4 Кодировщик скорости и блок слияния	26
5.5 Командные ветки и функция потерь	26
5.6 Конвейер сбора данных (collect_data.py)	28
5.7 Процесс обучения модели.....	31
5.8 Модуль инференса и безопасности (inference.py)	36
6 Описание структур данных	40
6.1 Формат датасета.....	40
6.2 BEV-тензор лидара.....	41
6.3 Метки навигационных команд.....	43
7 Тестирование модели в симуляционной среде.....	45
7.1 Методика тестирования.....	45
7.2 Количественные результаты.....	46
7.3 Качественный анализ поведения	47
7.4 Обобщение на незнакомые карты	48
7.5 Статус разработки и практическая значимость	49
8 Анализ ограничений и некорректного поведения модели	51
ЗАКЛЮЧЕНИЕ	53
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	55
ПРИЛОЖЕНИЕ А	58
ПРИЛОЖЕНИЕ Б.....	61
ПРИЛОЖЕНИЕ В	63
ПРИЛОЖЕНИЕ Г.....	69
ПРИЛОЖЕНИЕ Д	73
ПРИЛОЖЕНИЕ Е.....	76
ПРИЛОЖЕНИЕ Ж.....	81
ПРИЛОЖЕНИЕ И	88
ПРИЛОЖЕНИЕ К	89

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ТЕРМИНОВ

Используемые в работе определения, обозначения и сокращения — с раскрытой формулировкой:

- API (Application Programming Interface) — программный интерфейс приложения.
- BEV — Bird's Eye View — вид сверху (проекция облака точек)
- CIL — Conditional Imitation Learning — условное имитационное обучение
- CILRS — Conditional Imitation Learning with Speed Prediction
- CNN — Convolutional Neural Network — сверточная нейронная сеть
- GPU — Graphics Processing Unit — графический процессор
- GRU — Gated Recurrent Unit — вентильный рекуррентный блок
- LiDAR — Light Detection and Ranging — лидарный датчик дальности
- MLP — Multilayer Perceptron — многослойный перцептрон
- PID — Proportional–Integral–Derivative — ПИД-регулятор
- RL — Reinforcement Learning — обучение с подкреплением
- RGB — Red Green Blue — формат цветного изображения
- RNN — Recurrent Neural Network — рекуррентная нейронная сеть
- TM — Traffic Manager — менеджер трафика симулятора CARLA
- ВКР — Выпускная квалификационная работа
- НС — Нейронная сеть
- ПО — Программное обеспечение

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: Шрифт: не полужирный, английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

ВВЕДЕНИЕ

Автономное вождение остаётся одной из трудных и востребованных задач робототехники и искусственного интеллекта. Системы, которые ведут автомобиль без водителя, способны снизить аварийность на дорогах, разгрузить дорожную инфраструктуру и дать мобильность людям, которые не могут управлять машиной сами.

За последнее десятилетие область заметно продвинулась, однако в условиях настоящего городского трафика автономная навигация до конца не решена. Трудности связаны с тем, что приходится сводить вместе разнородные потоки данных камеры, лидара и радара, принимать решения при неполной информации и удерживать безопасное поведение в нештатных ситуациях.

Машинное обучение (Machine Learning) — направление искусственного интеллекта, в котором программа выводит закономерности прямо из данных и решает задачу тем точнее, чем больше примеров обработала; правила ей заранее не прописывают. Если в классическом алгоритме всю логику закладывает разработчик, то в модели машинного обучения внутренние параметры подбираются сами — во время обучения на размеченных либо неразмеченных данных.

Внутри машинного обучения отдельно выделяют глубокое обучение (Deep Learning) — модели на основе нейронных сетей с большим числом слоёв. Такая сеть сама выстраивает признаки по нарастанию сложности: от границ и текстур на изображении до целых объектов и дорожной сцены. Там, где признаки трудно придумать вручную, глубокие сети дали наибольший прирост качества.

Машинное и глубокое обучение применяют в компьютерном зрении, обработке естественного языка и речи, медицинской диагностике, рекомендательных и финансовых системах, робототехнике. Одно из самых требовательных направлений здесь — автономное вождение: модель в

Отформатировано: интервал Перед: 0 пт,
междустрочный, 1,5 строки

реальном времени разбирает показания нескольких датчиков и формирует управляющие сигналы. К этому классу задач относится и настоящая работа.

Конечной целью разработки систем автономного вождения является их применение на реальных дорогах общего пользования — в условиях настоящего городского трафика, с живыми участниками движения, реальными погодными условиями и непредсказуемыми дорожными ситуациями. Однако обучение и отладка таких систем непосредственно на реальном автомобиле сопряжены с высокими рисками для жизни и здоровья людей, значительными финансовыми затратами и юридическими ограничениями. Любая ошибка управления на реальной улице может привести к аварии.

Поэтому модели автономного вождения разрабатывают в два этапа. Сначала модель проектируют, обучают и проверяют в фотореалистичном симуляторе — в этой работе используется CARLA, — где без риска можно воспроизвести тысячи опасных ситуаций, собрать большой размеченный датасет и сколько угодно раз прогнать поведение агента. Затем обученную и проверенную модель переносят на настоящий автомобиль; этот шаг называют переносом «из симуляции в реальность» (sim-to-real transfer). Настоящая работа закрывает первый, базовый этап — создание и проверку работающей модели в симуляторе, которая позже может стать основой для автопилота на реальных дорогах.

Метод условного имитационного обучения (Conditional Imitation Learning, CIL), предложенный в работе Codevilla et al. [1], позволяет обучить нейронную сеть управлению транспортным средством на основе экспертных демонстраций, дополненных высокоуровневыми навигационными командами. Ключевое преимущество данного подхода — возможность управлять поведением агента на развилках: давать команды «повернуть налево», «повернуть направо», «ехать прямо» или «следовать по полосе». Целью преддипломной практики являлась разработка мультимодальной нейросетевой модели CILDriveNet для задачи автономной навигации

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

транспортного средства в симуляторе CARLA 0.9.14. В ходе практики были поставлены и решены следующие задачи:

- анализ существующих подходов к автономной навигации и обзор актуальных научных публикаций;
- выбор архитектуры нейронной сети и обоснование технологического стека;
- разработка мультимодальной модели CILDriveNet (RGB-камера + лидарный BEV-тензор + скорость + рекуррентный слой GRU);
- добавление рекуррентного модуля GRU для учёта краткосрочной истории движения и вспомогательной головы предсказания скорости;
- реализация конвейера автоматизированного сбора обучающих данных в симуляторе CARLA 0.9.14 с решением технических проблем API в синхронном режиме;
- обучение модели на собранном датасете (карта Town10HD, 25 000 кадров) с функцией потерь branch-gated CIL loss и вспомогательным предсказанием скорости~~20 000 кадров~~);
- количественная оценка модели на валидационной выборке;
- разработка скрипта инференса и тестирования в реальном времени с трафиком из 30 NPC-машин, двухуровневой защитой от столкновений и автоматическим respawn~~двухуровневой защитой от столкновений~~.

Объект исследования — система управления транспортным средством на основе методов глубокого обучения.

Предмет исследования — архитектуры нейронных сетей с рекуррентной памятью и методы обучения для задачи условного имитационного обучения автономного вождения.

Практический результат работы — действующий прототип системы автономной навигации: модель CILDriveNet обучена и проверена в реальном времени в симуляторе CARLA 0.9.14 с трафиком из 30 NPC-машин. Написанные модули (сбор данных, обучение, инференс) вместе образуют замкнутый исследовательский конвейер. Его можно взять за базовую линию

Отформатировано: Отступ: Слева: 0 см, Первая строка: 1,25 см, междустрочный, 1,5 строки

Отформатировано: Отступ: Слева: 0 см, Первая строка: 1,25 см, междустрочный, 1,5 строки

Отформатировано: Отступ: Слева: 0 см, Первая строка: 1,25 см, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: междустрочный, 1,5 строки

(baseline) при подготовке выпускной квалификационной работы и применять в учебных курсах по глубоким нейросетям и автономному транспорту.

1 Анализ предметной области

1.1 Машинное и глубокое обучение

отформатировано: Шрифт: (по умолчанию) Times New Roman, 14 пт, полужирный, Цвет шрифта: Авто

Машинное обучение объединяет методы построения математических моделей, которые находят закономерности в данных и переносят их на новые, ранее не встречавшиеся примеры. Формально обучение — это подбор параметров модели, при котором функция потерь на обучающей выборке достигает минимума. По постановке задачи выделяют три вида: обучение с учителем работает с размеченными данными, обучение без учителя ищет структуру в неразмеченных, а обучение с подкреплением опирается на взаимодействие со средой и систему вознаграждений.

Глубокое обучение — направление машинного обучения, основанное на искусственных нейронных сетях с большим числом скрытых слоёв. Каждый слой выполняет нелинейное преобразование входных данных, а их последовательность формирует иерархию признаков возрастающей сложности. Для обработки изображений применяются свёрточные нейронные сети (CNN), для последовательностей и временных рядов — рекуррентные нейронные сети (RNN, в том числе LSTM и GRU). Обучение выполняется методом обратного распространения ошибки с применением алгоритмов стохастической оптимизации.

Методы глубокого обучения находят применение в компьютерном зрении, обработке естественного языка, распознавании речи, биоинформатике, рекомендательных системах и робототехнике. В задачах автономного вождения нейронные сети применяются для восприятия дорожной сцены, предсказания траекторий участников движения и непосредственного формирования управляющих воздействий (end-to-end подход). Именно сочетание свёрточных и рекуррентных архитектур, рассматриваемое в настоящей работе, позволяет учитывать как пространственную структуру сцены, так и динамику её изменения во времени.

Преимущество глубокого обучения перед классическими алгоритмами в задачах автономного вождения состоит в том, что нейронная сеть

самостоятельно извлекает признаки непосредственно из «сырых» данных сенсоров (пикселей изображения, облака точек лидара) и формирует управляющее решение без необходимости вручную программировать правила для каждой дорожной ситуации. Это особенно важно для реального мира, где число возможных сценариев практически бесконечно и заранее описать их все правилами невозможно.

От модели, которую готовят к выходу на реальные дороги, прежде всего требуется обобщение (generalization) — устойчивая работа в условиях, которых не было при обучении: на незнакомых улицах, при другой погоде и плотности движения. Этого добиваются предобучением на крупных наборах данных (например, ImageNet), аугментацией примеров и сбором разнообразной выборки. На этих принципах построена и модель настоящей работы.

1.2 Задача автономной навигации транспортного средства

Задача автономной навигации транспортного средства ставится так: система в реальном времени обрабатывает данные сенсоров и вырабатывает управляющие воздействия (угол поворота руля, дроссель, тормоз), чтобы безопасно довести транспортное средство до целевой точки и не нарушить правила дорожного движения [2].

Задача традиционно декомпозируется на подзадачи: восприятие (perception), предсказание (prediction) и планирование (planning). Системы с глубоким обучением объединяют эти компоненты в одну сквозную (end-to-end) архитектуру, и тогда между модулями ничего не теряется [3].

Основные трудности — это обобщение на новые условия и надёжность в критических ситуациях. К ним добавляется дисбаланс данных: в естественной поездке прямых участков набирается заметно больше, чем поворотов.

Отформатировано: Обычный, По ширине, Отступ:
Первая строка: 1,25 см

Отформатировано: интервал Перед: 0 пт,
междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт,
междустрочный, 1,5 строки

1.3 Обзор подходов к решению задач

задач

Для автономной навигации сегодня применяют три парадигмы машинного обучения:

Имитационное обучение (Behavioral Cloning, BC) — модель учится воспроизводить поведение эксперта-человека. Главная проблема — накопление ошибок (covariate shift) [9].

Обучение с подкреплением (Reinforcement Learning, RL) — агент учится, взаимодействуя со средой. Подход неустойчив, а функцию вознаграждения сложно спроектировать [10].

Условное имитационное обучение (CIL) — расширение BC: модели дополнительно подаётся высокоуровневая навигационная команда, что снимает неоднозначность на перекрёстках [1]. Данный подход выбран в настоящей работе.

1.4 Метод условного имитационного обучения (CIL)

Метод CIL предложили Codevilla et al. в 2018 году [1]. Основная идея — архитектура с несколькими «ветками» (branches), каждая отвечает за одну навигационную команду. При обучении градиент идёт только через ту ветку, что соответствует записанной команде (branch-gating).

В работе реализованы четыре команды: FOLLOW (следовать по полосе), LEFT (повернуть налево), RIGHT (повернуть направо), STRAIGHT (ехать прямо через перекрёсток). Команда в реальном времени выбирается по геометрии дороги — на основе путевых точек (waypoints) CARLA.

Модель CILDriveNet, разработанная в этой работе, расширяет оригинальный CIL: добавлен лидар в виде BEV-тензора, а кодировщиком изображений служит предобученная ResNet-18.

1.5 Рекуррентные нейронные сети в задачах управления

Стандартный метод CIL обрабатывает каждый кадр независимо: модель не располагает памятью о предыдущих состояниях. Это порождает ряд

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

отформатировано: Шрифт: (по умолчанию) Times New Roman, 14 пт, полужирный, Цвет шрифта: Авто

затруднений — при входе в поворот (нет информации о начале манёвра), при торможении перед препятствием (один кадр не отражает динамику сближения) и при смене полосы (отсутствует контекст предшествующего движения).

Решением является применение рекуррентных нейронных сетей (RNN), которые передают информацию между шагами через скрытое состояние и тем самым учитывают временной контекст. Наибольшее распространение получили архитектуры LSTM (Long Short-Term Memory) и GRU (Gated Recurrent Unit), снабжённые механизмами вентиляей (gates) для управления потоком информации и борьбы с проблемой затухающего градиента.

В работе выбран блок GRU: он проще и дешевле в вычислениях, чем LSTM, а по качеству ему не уступает. GRU стоит в CILDriveNet между блоком слияния признаков и командными ветками. На вход идёт последовательность из четырёх кадров подряд, а при инференсе скрытое состояние переносится от тика к тика. Благодаря этому модель помнит недавнюю историю движения и в динамичных ситуациях действует устойчивее.

Работа базового рекуррентного слоя на шаге t описывается соотношением (1):

$$h_t = f(W_x \cdot x_t + W_h \cdot h_{t-1} + b) \quad (1)$$

где x_t — вход текущего шага; h_{t-1} — скрытое состояние предыдущего шага; f — нелинейная функция активации. Однако базовые RNN плохо обучаются на длинных последовательностях из-за затухания и взрыва градиента: при обратном распространении ошибки через множество шагов произведение производных либо стремится к нулю, либо неограниченно растёт.

Эту проблему решают вентиляльные архитектуры. LSTM вводит ячейку памяти и три вентиля, тогда как GRU (Cho et al., 2014) упрощает схему до двух вентиляей при сопоставимом качестве, объединяя ячейку памяти и скрытое состояние. Работа GRU на шаге t описывается соотношениями (2)–(5):

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad (2)$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \quad (3)$$

$$g_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t]) \quad (4)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot g_t \quad (5)$$

Где: σ — сигмоида, $\odot$ — поэлементное умножение. Вентиль обновления z_t определяет, в какой мере новое состояние перенимает прошлое: при z_t , близком к нулю, сеть почти полностью сохраняет предыдущее состояние, что обеспечивает удержание долгосрочного контекста. Вентиль сброса r_t управляет тем, насколько прошлое состояние влияет на кандидата g_t , позволяя сети при необходимости забывать неактуальную историю. Меньшее число параметров GRU по сравнению с LSTM даёт преимущество при инференсе в реальном времени и снижает риск переобучения на ограниченной выборке, что и обусловило выбор именно этой архитектуры в настоящей работе.

Внутреннее устройство блока GRU поясняет рисунок 1. На вход поступают текущий элемент последовательности x_t и скрытое состояние предыдущего шага h_{t-1} . Два вентиля управляют потоком информации. Вентиль сброса r_t (сигмоида σ) определяет, какая часть прошлого состояния участвует в формировании кандидата нового состояния $\tilde{h}_t$ (через гиперболический тангенс $\tanh$). Вентиль обновления z_t задаёт пропорцию, в которой итоговое состояние h_t складывается из прошлого состояния h_{t-1} и кандидата $\tilde{h}_{t-1}$; на схеме это реализуют операции поэлементного умножения ($\times$) и взвешивания $(1 - z_t)$. По сравнению с LSTM, где используются три вентиля и отдельная ячейка памяти, GRU объединяет память и скрытое состояние, что уменьшает число параметров без существенной потери качества и ускоряет обучение.

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

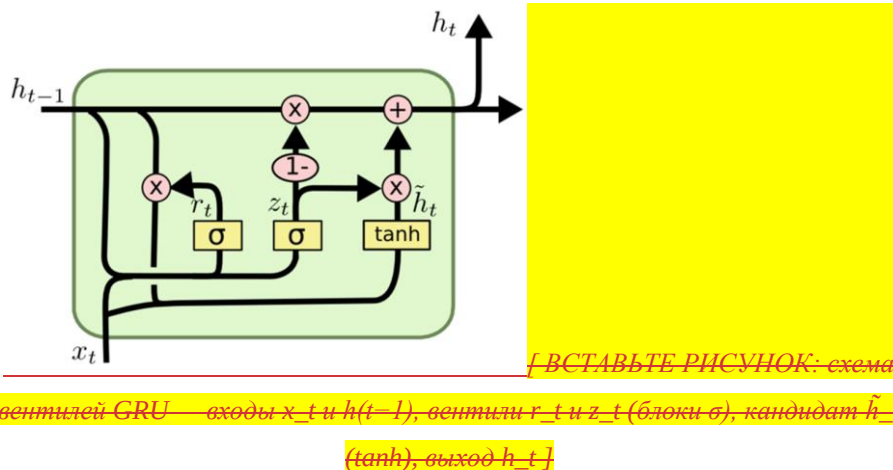


Рисунок 1 — Схема вычислений в блоке GRU (вентили обновления и сброса)

1.6 Моделирование в виртуальных средах1.4Моделирование в виртуальных средах

Для обучения и тестирования моделей широко применяют фотореалистичные симуляторы — они дают возможность безопасно собирать данные в разных условиях. Из распространённых стоит назвать: CARLA [5], SUMO, Waymo Simulation, AirSim.

В работе используется CARLA версии 0.9.14 — открытый симулятор с синхронным режимом, встроенным TrafficManager и широким набором сенсоров (RGB-камера, лидар, радар). Данные собирались на карте Town10HD — это городская среда с перекрёстками, кольцевыми развязками и светофорами, которая даёт достаточное разнообразие сценариев вождения.

Отформатировано: По левому краю, Отступ: Первая строка: 0 см, интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки



Рисунок 2 — Среда симулятора CARLA 0.9.14, карта Town10HD

На рисунке 2 показан общий вид городской среды Town10HD, на которой собирались обучающие данные. Карта включает участки многополосного движения, регулируемые и нерегулируемые перекрёстки, кольцевые развязки и пешеходные переходы. Благодаря такому разнообразию дорожных ситуаций выборка получается репрезентативной для всех четырёх команд (FOLLOW, LEFT, RIGHT, STRAIGHT), а набор сценариев — статистически сбалансированным.

Отформатировано: Отступ: Первая строка: 1,25 см, интервал после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

2 Постановка задачи и требования к системе

2.1 Цели и задачи разработки

Целью разработки является мультимодальная нейросетевая модель, которая управляет транспортным средством в симулированной городской среде по навигационным командам. Модель должна работать в реальном времени на GPU NVIDIA с поддержкой CUDA. Для достижения этой цели поставлены следующие задачи:

- разработать архитектуру нейронной сети с тремя входными потоками: изображение с камеры, облако точек лидара в BEV-проекции, текущая скорость;
- добавить рекуррентный модуль GRU для учёта истории движения (последовательность из 4 кадров);
- реализовать конвейер автоматизированного сбора обучающих данных в синхронном режиме симулятора CARLA 0.9.14;
- обеспечить сохранение синхронизированных данных сенсоров и меток управления в едином датасете;
- разработать скрипт обучения с реализацией функции потерь с ветковым затвором (branch-gated CIL loss) и вспомогательным предсказанием скорости;
- реализовать скрипт инференса с двухуровневой защитой от столкновений (sensor.other.obstacle + LIDAR), трафиком из 30 NPC-машин и автоматическим respawn.LIDAR).

2.2 Функциональные требования

Таблица 1 — Функциональные требования к системе

№	Требование	Описание
Ф-01	Обработка изображения	Кадры 224×224, нормализация по ImageNet-статистикам
Ф-02	Обработка лидара	BEV-тензор 5×200×200, покрытие ±25 м

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: Отступ: Слева: 0 см, Первая строка: 1,25 см, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Продолжение таблицы 1

№	Требование	Описание
Ф-03	Учёт скорости	Скорость нормализована в $[0, 1]$, подаётся отдельным вектором
Ф-04	Навигационные команды	4 команды: FOLLOW, LEFT, RIGHT, STRAIGHT
Ф-05	Временная память	GRU обрабатывает последовательность из 4 кадров
Ф-06Ф-05	Выходные данные	$\text{steer} \in [-1, 1]$ (tanh), $\text{throttle} \in [0, 1]$ (sigmoid)
Ф-07Ф-06	Безопасность	Двойное обнаружение препятствий: obstacle sensor + LIDAR
Ф-08Ф-07	Запись датасета	Синхронное PNG + NPY + CSV за каждый тик симулятора

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

отформатировано: русский

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Добавлено примечание ([ДГ1]):

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

2.3 Требования к данным и аппаратному обеспечению

Требования к обучающему датасету: не менее 20 000 (собрано 25 000)45* 000 кадров, собранных в симуляторе CARLA на карте Town10HD; сбалансированное распределение навигационных команд (не менее 8% кадров для LEFT, RIGHT, STRAIGHT); скорость в диапазоне 5–30 км/ч.

Минимальные аппаратные требования: CPU с частотой не менее 2.5 ГГц, 16 ГБ ОЗУ, GPU NVIDIA с видеопамятью не менее 4 ГБ и поддержкой CUDA 11+.

Таблица 2 — Аппаратная конфигурация рабочей станции

Компонент	Характеристика
Процессор (CPU)	AMD Ryzen 5 5600H, 6 ядер / 12 потоков, 3.3 ГГц (до 4.2 ГГц в режиме Turbo)

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Оперативная память (RAM)	16 ГБ DDR4, 3200 МГц
--------------------------	----------------------

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Продолжение таблицы 2

Компонент	Характеристика
Графический процессор (GPU)	NVIDIA GeForce RTX 3050 Laptop, 4 ГБ GDDR6, CUDA 11.8
Накопитель	SSD NVMe, 477 ГБ
Операционная система	Windows 11 (64-bit)
Программная среда	Python 3.7, PyTorch 1.13, CUDA Toolkit 11.8

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: Отступ: Первая строка: 0 см

3 Сравнение с аналогами

3.1 CILRS — Conditional Imitation Learning with Speed Prediction

CILRS [2] — расширение оригинальной CIL архитектуры (Codevilla et al., 2019). Ключевое нововведение — отдельная ветка предсказания скорости, частично решающая проблему «инертного агента». Преимущества: апробированная архитектура. Недостатки: одномодальный вход (только RGB), отсутствие лидара снижает надёжность обнаружения препятствий.

3.2 Learning by Cheating (LBC) и Roach

Learning by Cheating [3] — двухэтапный подход: сначала обучается «привилегированный» агент с полным доступом к информации симулятора, затем методом дистилляции обучается сенсорный агент. Roach [4] использует агент на основе RL как учителя. Оба подхода показывают высокие результаты, но требуют значительных вычислительных ресурсов.

3.3 TransFuser

TransFuser [6] — архитектура на трансформерах для многомодального обучения с объединением данных камеры и лидара через cross-attention. Лидер CARLA Leaderboard 2021. Недостатки: требует GPU > 16 ГБ для обучения, сложна в реализации с нуля.

3.4 Обоснование собственной разработки

Таблица 3 — Сравнение систем автономной навигации

Критерий	CILRS	LBC/Roach	TransFuser	CILDriveNet (наша)
Входные данные	RGB	RGB+при-вил.	RGB+LiDAR	RGB+LiDAR+Speed
Архитектура	CNN+MLP	CNN+RL	Transformer	ResNet-18+CNN+GRU ResNet18+CNN+MLP

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Код поля изменен

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: По ширине, интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Продолжение таблицы 3

Критерий	CILRS	LBC/Roach	TransFuser	CILDriveNet (наша)
<u>Рекуррентная память</u>	<u>Нет</u>	<u>Нет</u>	<u>Нет</u>	<u>GRU (4 кадра)</u>
Требования к GPU	Низкие	Средние	Высокие	Средние
Сложность реализ.	Низкая	Высокая	Высокая	Средняя
Учёт скорости	Да	Неявно	Да	Да (encoder)
<u>Предсказание скорости</u>	<u>Да</u>	<u>Нет</u>	<u>Нет</u>	<u>Да (aux head)</u>
LiDAR BEV	Нет	Нет	Да	Да
Obstacle Sensor	Нет	Нет	Нет	Да

CILDriveNet — компромисс между CILRS и TransFuser: добавлены лидарный BEV и sensor.other.obstacle чтобы надёжнее обнаруживать препятствия без чрезмерной нагрузки на вычисления. ResNet-18 быстрее сходится на небольшой выборке. Дополнительно введены рекуррентный модуль GRU (учёт истории из 4 кадров) и вспомогательная голова предсказания скорости (Speed Head), что отличает CILDriveNet от CILRS и одиночно-кадровых аналогов.

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, междустрочный, 1,5 строки

3.5 Рекуррентные и гибридные модели в смежных задачах

Применение рекуррентных и свёрточных сетей в связке исследовалось неоднократно, однако большинство близких работ решают задачу предсказания траектории, а не прямого формирования управляющих сигналов. Ниже рассмотрены наиболее показательные решения с акцентом на различие в постановке задачи.

Social-LSTM (Alahi et al., 2016) — одна из ранних работ, применивших рекуррентные сети к движению в городской среде. Для каждого пешехода обучается отдельная LSTM, а взаимодействие между агентами моделируется через объединение скрытых состояний соседей. Работа подтвердила, что рекуррентное состояние эффективно кодирует динамику движения. В отличие от неё, разрабатываемая модель формирует непосредственно управляющие воздействия и не требует явного моделирования соседних агентов.

Гибридные модели вида CNN+GRU, применяемые для прогноза движения объектов, подтверждают эффективность связки свёртки и вентильной рекуррентности: свёрточная часть извлекает пространственные признаки, а рекуррентная моделирует их изменение во времени. Такие модели, однако, нацелены на восприятие и прогноз чужих траекторий, тогда как разрабатываемая модель управляет собственным транспортным средством по навигационной команде.

По сравнению с трансформерными решениями (например, TransFuser), использующими cross-attention для слияния камеры и лидара, CILDriveNet применяет более лёгкую конкатенацию признаков и рекуррентный слой. Это снижает требования к видеопамяти при сохранении учёта временного контекста. Таким образом, ключевое отличие разработанной модели от рассмотренных аналогов — сочетание трёх свойств в одной компактной архитектуре: прямое управление через командные ветки, мультимодальный вход с лидарным BEV и рекуррентная память на основе GRU.

4 Выбор платформы и инструментальных средств

4.1 Симулятор CARLA

CARLA (Car Learning to Act) [5] — открытый фотореалистичный симулятор городского вождения, разработанный компанией Intel Labs и Universitat Autònoma de Barcelona. Предоставляет Python API для управления симуляцией, автоматического пилота (TrafficManager), погодных условий и набора сенсоров.

Ключевые возможности CARLA 0.9.14, использованные в работе: синхронный режим — мир обновляется только по явной команде `world.tick()`, благодаря чему данные сенсоров детерминированно соответствуют друг другу; TrafficManager — встроенный менеджер трафика для управления поведением NPC; карта Town10HD — городская среда с кольцевыми развязками и светофорами; поддержка RGB-камеры, лидарного датчика Ray Cast и датчика препятствий `sensor.other.obstacle`.

Альтернативой рассматривался симулятор SUMO, однако он не обеспечивает нативную поддержку лидарных сенсоров и фотореалистичного рендеринга.

4.2 Язык программирования и фреймворк

Язык Python 3.7 выбран в силу наличия официального CARLA Python API только для версий 3.6–3.8, а также развитой экосистемы библиотек глубокого обучения.

Фреймворк PyTorch выбран как основной инструмент. По сравнению с TensorFlow обеспечивает более гибкий динамический граф вычислений. Дополнительные библиотеки: NumPy, OpenCV, Pandas, TensorBoard, tqdm.

4.3 Предобученная CNN-архитектура ResNet-18

ResNet-18 [7] — остаточная сверточная сеть с 18 слоями, предобученная на ImageNet. Используется в качестве кодировщика изображений: последний FC-слой удаляется, выходом служит 512-мерный вектор признаков. Низкоуровневые признаки (края, текстуры, цвета) переносятся из ImageNet,

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

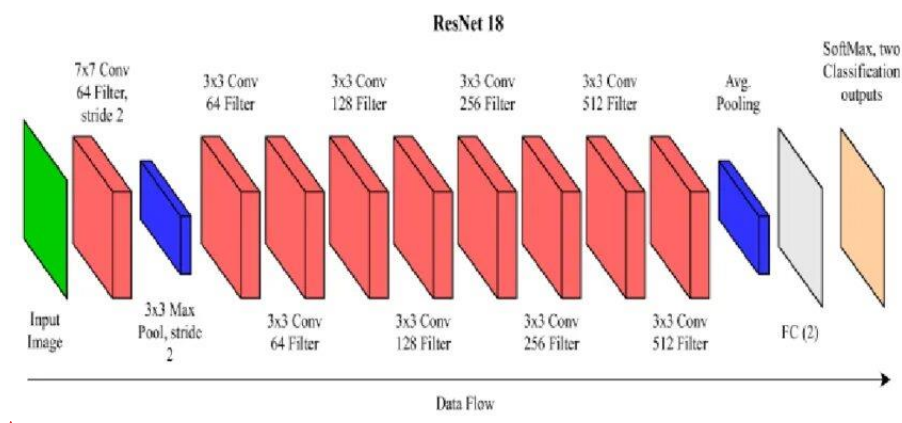
Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

что важно при небольшом собственном датасете. Дифференцированные темпы обучения: backbone 3×10^{-5} , остальные слои 3×10^{-4} .

Структура сети ResNet-18 приведена на рисунке 3. Входное изображение проходит через начальный свёрточный слой 7×7 (64 фильтра, шаг 2) и слой подвыборки MaxPool 3×3 , после чего следуют четыре группы остаточных блоков с возрастающим числом фильтров — 64, 128, 256 и 512. Каждый остаточный блок содержит два свёрточных слоя 3×3 и снабжён сквозным соединением (residual connection), которое складывает вход блока с его выходом; именно эти соединения позволяют обучать глубокую сеть, не сталкиваясь с затуханием градиента. Завершают сеть слой глобального усреднения Average Pooling и полносвязный слой. В настоящей работе последний полносвязный слой удалён, а 512-мерный вектор признаков с выхода Average Pooling используется как признаковое представление изображения, подаваемое в блок слияния.



отформатировано: Шрифт: не курсив, не выделение цветом

ВСТАВЬТЕ РИСУНОК: структура ResNet-18 — Conv 7×7 , MaxPool, остаточные блоки 3×3 (64/128/256/512 фильтров), Average Pooling, FC

Рисунок 3 — Структура свёрточной сети ResNet-18

4.4 Лидарный датчик и BEV-представление

Лидарный датчик Ray Cast сконфигурирован: 32 канала, дальность 40 м, 56 000 точек/с, 20 Гц. Монтируется на крыше транспортного средства на высоте 2.4 м. Облако точек преобразуется в BEV-тензор $5 \times 200 \times 200$

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

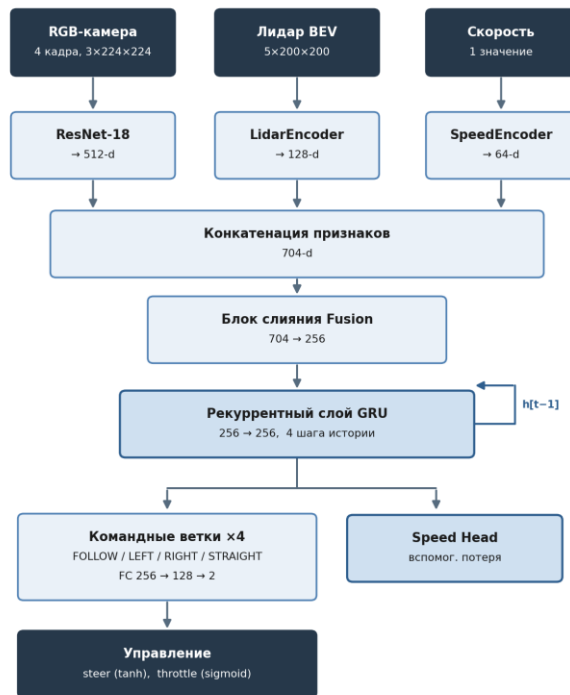
(разрешение 0.25 м/пиксель, покрытие ± 25 м). Первые 4 канала — высотные срезы ($z \in [-3, 2]$ м), пятый — нормализованная интенсивность. BEV-формат удобен для свёрток и позволяет обнаруживать препятствия и другие машины.

5 Описание разработки программного обеспечения

5.1 Общая архитектура модели CILDriveNet

Архитектура CILDriveNet — мультимодальная нейронная сеть с рекуррентной памятью, объединяющая три независимых потока. На вход подаётся последовательность из $T=4$ последовательных кадров камеры, текущий лидарный BEV-тензор и текущая скорость. Признаки последовательности обрабатываются рекуррентным модулем GRU, накапливающим краткосрочную историю движения. Выход — четыре пары~~Выход — четыре пары~~ (steer, throttle), по одной на каждую навигационную команду. В режиме инференса выбирается пара, соответствующая текущей команде планировщика.

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки



Отформатировано: интервал Перед: 0 пт, после: 0 пт

При обучении градиент идёт только через ветку текущей команды (branch-gating)

Рисунок 4 — Архитектура модели CILDriveNet

На рисунке 4 показана общая схема модели CILDriveNet. Три кодировщика (изображения, лидарного BEV-тензора и скорости) формируют независимые признаковые векторы, которые конкатенируются в блоке слияния (Fusion). Объединённое представление за каждый из $T=4$ кадров последовательно подаётся в рекуррентный модуль GRU, скрытое состояние которого передаёт краткосрочную историю движения; выход последнего шага GRU поступает~~конкатенируются в блоке слияния (Fusion)~~. ~~Полученное общее представление сцены поступает~~ одновременно во все четыре командные ветки, однако градиент при обучении распространяется только через ветку, соответствующую текущей навигационной команде (см. рисунок 5).

Общее число параметров модели: ~~~11.5 млн~~~~~11.3 млн~~ (из них ~11.2 млн — ResNet-18 backbone). Реализация: scripts/model.py (листинг приведён в приложении А).

5.2 Модуль обработки изображений с камеры

Входное изображение (640×480 , BGR из CARLA) проходит предобработку: преобразование BGR→RGB, масштабирование до 224×224 , нормализация по ImageNet: вычитание среднего $[0.485, 0.456, 0.406]$ и деление на std $[0.229, 0.224, 0.225]$.

При обучении применяются аугментации: горизонтальное отражение (с зеркалированием руля и переключением LEFT↔RIGHT), цветовое дрожание ($\alpha \in [0.8, 1.2]$, $\beta \in [-20, 20]$), изменение яркости в HSV-пространстве, случайный LIDAR dropout 5%.

5.3 Модуль обработки данных лидара (LidarEncoder)

LidarEncoder — лёгкая сверточная сеть: три блока Conv2d (stride=2) + BatchNorm + ReLU (каналы $5 \rightarrow 32 \rightarrow 64 \rightarrow 128$), AdaptiveAvgPool2d(1), Flatten. Выход: 128-мерный вектор признаков.

отформатировано: Шрифт: 9 пт, английский (США)

Отформатировано: междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Функция `lidar_to_bev()` (`utils.py`, листинг — приложение Б) преобразует облако точек (N, 4) в BEV-тензор: координаты x, y → позиция пикселя; z → индекс высотного канала; интенсивность → канал 5.

5.4 Кодировщик скорости и блок слияния

SpeedEncoder принимает скорость, нормализованную делением на `MAX_SPEED_KMH = 60` км/ч. MLP: два слоя (1→64→64) с ReLU. Fusion конкатенирует три потока [704-d] и пропускает через FC(704→512, ReLU) → Dropout(0.3) → FC(512→256, ReLU).

Рекуррентный модуль GRU (Gated Recurrent Unit) добавлен после блока слияния и составляет ключевое отличие CILDriveNet от базового метода CIL. Входной и скрытый размеры — 256, один слой. При обучении на вход GRU подаётся последовательность из T=4 fusion-векторов размерности (T, B, 256); используется выход последнего шага $x = \text{gru_out}[-1]$ размерности (B, 256), к которому применяется Dropout(0.3). При инференсе модель работает по одному кадру (T=1), а скрытое состояние h передаётся между тиками симулятора и сбрасывается после аварии. Благодаря GRU модель учитывает краткосрочную историю движения: начало манёвра в повороте, динамику сближения с препятствием и контекст смены полосы.

5.5 Командные ветки и функция потерь

Четыре командные ветки (ModuleList): FC(256→128, ReLU) → FC(128→2). Активация: tanh для руля, sigmoid для throttle.

Функция потерь Branch-gated CIL Loss с усиленным весом руля и дополнительным коэффициентом для поворотных команд рассчитывается по формуле (6):

$$L_{CIL} = w_{s_c} \cdot w_{c_c} \cdot \text{SmoothL1}(\hat{s}_c, s_c) + \text{MSE}(\hat{t}_c, t_c) \quad (6)$$

Где:

c — индекс активной команды (из файла `labels.csv`);

($\hat{s}_c$, $\hat{t}_c$ — предсказания ветки c

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, междустрочный, 1,5 строки

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

Отформатировано: Отступ: Первая строка: 1,25 см, междустрочный, 1,5 строки

s, t — эталонные steer и throttle

$w_s = 8,0$ — вес руля (критичнее throttle)

$w_c = 1 + \text{turn_mask}$ ($\times 2$ для LEFT/RIGHT/STRAIGHT)

Отформатировано: междустрочный, 1,5 строки

Вспомогательная голова предсказания скорости (Speed Head).

отформатировано: русский

Дополнительно к командным веткам модель содержит вспомогательную голову, предсказывающую нормализованную скорость по визуальным признакам текущего кадра ($\text{cam_f} + \text{lidar_f}$, 640-d), но не по самой скорости — иначе задача стала бы тривиальной. Структура: $\text{FC}(640 \rightarrow 128, \text{ReLU}) \rightarrow \text{FC}(128 \rightarrow 1) \rightarrow \text{Sigmoid}$.

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: Цвет шрифта: черный, русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

отформатировано: русский

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Вспомогательная и суммарная функции потерь определяются формулами (7) и (8):

$$L_{\text{speed}} = \text{MSE}(\hat{v}, v) \quad (7)$$

$$L_{\text{total}} = L_{\text{CIL}} + 0,5 \cdot L_{\text{speed}} \quad (8)$$

Где: $\hat{v}$ — предсказанная нормализованная скорость; v — её эталонное значение; коэффициент 0,5 задаёт вклад вспомогательной потери в суммарную.

Цель вспомогательной головы — научить модель оценивать скорость из визуального контекста, что косвенно улучшает поведение при торможении перед препятствиями и перекрёстками.

Оптимизатор: AdamW, $\text{lr_backbone} = 3 \times 10^{-5}$, $\text{lr_head} = 3 \times 10^{-4}$, $\text{weight_decay} = 10^{-4}$. Планировщик: CosineAnnealingWarmRestarts ($T_0=20$, $T_{\text{mult}}=2$). Ранняя остановка: $\text{patience} = 20$ эпох. Максимум: 80 эпох.

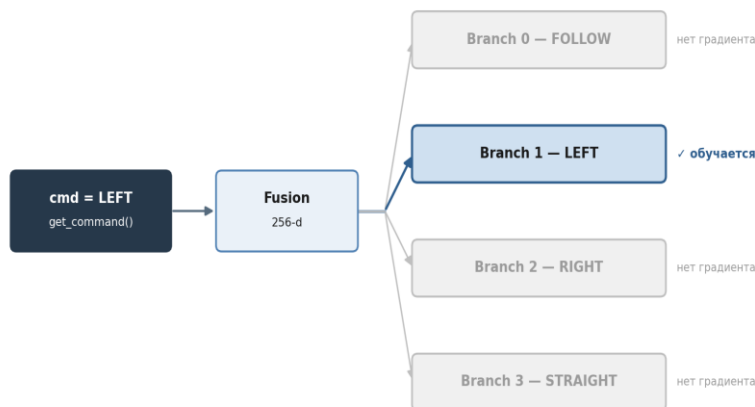


Рисунок 5 — Схема ветвления градиента в CIL

На рисунке 5 проиллюстрирован принцип ветвления градиента (branching). Несмотря на то, что прямой проход модели вычисляет предсказания во всех четырёх ветках, обратный проход на этапе обучения активирует только ту ветку, индекс которой соответствует записанной экспертной команде. Такой подход исключает интерференцию обучающих сигналов между ветками и позволяет каждой из них специализироваться на конкретном типе манёвра.

5.6 Конвейер сбора данных (collect_data.py)

Сбор данных выполнен на карте Town10HD (25 000 кадров)(20-000 кадров) с ТМ-автопилотом (листинг скрипта collect_data.py — приложение В). ТМ обрабатывает перекрёстки, светофоры и кольцевые развязки без вмешательства разработчика, за счёт этого датасет покрывает разные дорожные ситуации.

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, одинарный

Отформатировано: интервал после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

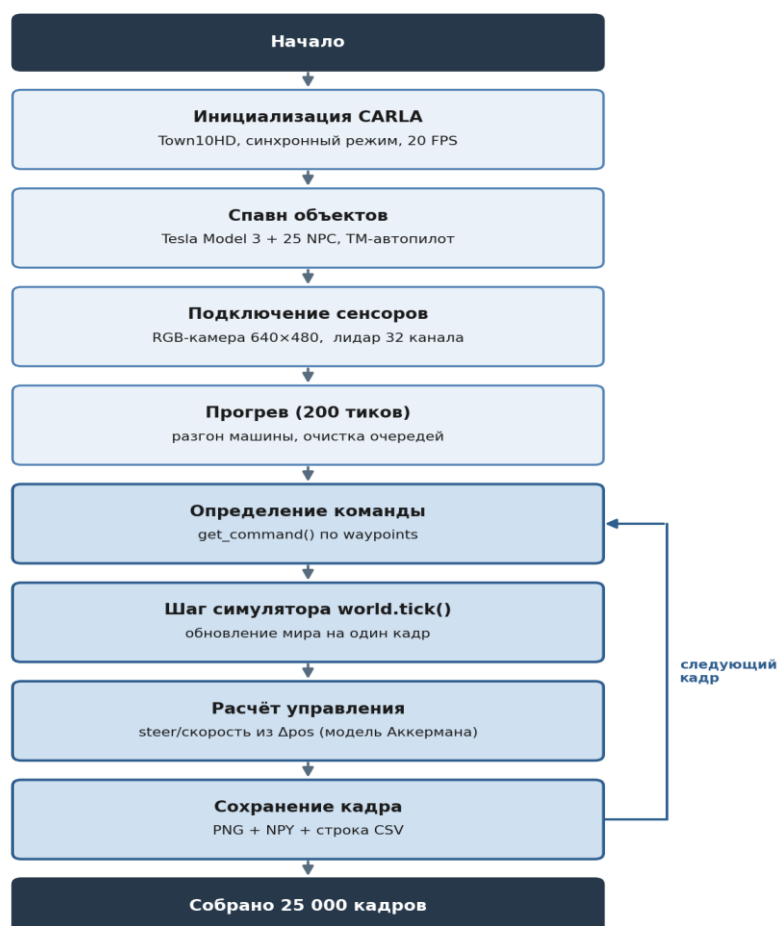


Рисунок 6 — Конвейер сбора данных

Конвейер, представленный на рисунке 6, реализует синхронный сбор данных: на каждом тике симулятора ($\Delta t = 0,1$ с, 10 Гц) выполняется однократный вызов `world.tick()`, после чего из очередей сенсоров извлекаются изображение с RGB-камеры и облако точек лидара, рассчитываются управляющие сигналы по модели Аккермана и сохраняются файлы в форматах PNG, NPY и строка в файле `labels.csv`. При такой схеме сенсорные данные и метки управления относятся ровно к одному кадру.

Ключевая техническая особенность (CARLA 0.9.14 bug): в синхронном режиме `vehicle.get_control()` и `vehicle.get_velocity()` возвращают нули для ТМ-управляемых машин. Причина — `TrafficManager` с включённым `hybrid_physics_mode` не применяет физику к эго-машине. Решение: `tm.set_hybrid_physics_mode(False)` + вычисление управляющих сигналов из дельты позиции (Δp_{pos} , $\Delta \psi_{\text{yaw}}$) по модели Аккермана:

Скорость рассчитывается как $v = |\Delta p_{\text{pos}}| / \Delta t \cdot 3,6$ (км/ч), угловая скорость рыскания $\omega = \Delta \psi / \Delta t$, угол поворота передних колёс по модели Аккермана $\delta = \arctan(\omega \cdot L / v)$, нормированное управление рулём $\text{steer} = \text{clip}(\delta / \delta_{\text{max}}, -1, 1)$, где $\Delta t = 0,1$ с — шаг симуляции, $L = 2,875$ м — колёсная база Tesla Model 3, $\delta_{\text{max}} = 1,22$ рад ($\approx 70^\circ$) — максимальный угол поворота колёс.

Исправление скорости ТМ. Вторая особенность CARLA 0.9.14: при `tm.vehicle_percentage_speed_difference(vehicle, 0)` машина строго соблюдает знак ограничения скорости. В Town10HD ряд зон имеет ограничение 5–10 км/ч, из-за чего эго-машина движется со скоростью 2–3 км/ч, и собранный датасет вырождается. Решение — задать `tm.vehicle_percentage_speed_difference(vehicle, -50)`: машина движется на 50 % быстрее знака, обеспечивая стабильные 25–30 км/ч.

Параметры сбора данных: 25 000 кадров при 20 FPS в синхронном режиме; 25 NPC-машин под управлением ТМ; прогрев 200 тиков перед началом записи; детектор застревания (при скорости $< 1,5$ км/ч в течение 15 с автопилот перезапускается без телепортации); смена погодных условий каждые 600 тиков в пределах дневного диапазона. В файл `labels.csv` добавлена колонка `cam (C/L/R)`, указывающая камеру кадра, — она необходима для построения корректных временных последовательностей при обучении GRU.

```

Диагностика первых 5 сохранённых кадров:
[кадр 0] steer=-0.0001 thr=0.450 brk=0.000 spd=23.18 km/h cmd=STRAIGHT lidar=✓
[кадр 1] steer=-0.0000 thr=0.450 brk=0.000 spd=22.96 km/h cmd=STRAIGHT lidar=✓
[кадр 2] steer=-0.0000 thr=0.450 brk=0.000 spd=22.77 km/h cmd=STRAIGHT lidar=✓
[кадр 3] steer=-0.0000 thr=0.450 brk=0.000 spd=22.56 km/h cmd=STRAIGHT lidar=✓
[кадр 4] steer=+0.0000 thr=0.450 brk=0.000 spd=22.37 km/h cmd=STRAIGHT lidar=✓

```

Рисунок 7 — Диагностический вывод сбора данных

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал После: 0 пт, после: 0 пт, междустрочный, одинарный

Отформатировано: интервал после: 0 пт, междустрочный, 1,5 строки

По первым кадрам контролируется корректность вычисленных значений: ненулевое значение скорости (`speed_kmh`) и значимая динамика угла поворота руля (`steer`) показывают, что вычисление через модель Аккермана даёт осмысленные значения вместо нулей, которые возвращает CARLA API.

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

5.7 Процесс обучения модели

Обучение выполнено с использованием скрипта `train.py` (листинг — приложение Г). Датасет разделён последовательно: первые 85% кадров — обучение, последние 15% — валидация (исключение утечки данных между временно близкими кадрами). Для балансировки применён `WeightedRandomSampler`: поворотные кадры ($|\text{steer}| > 0.06$) $\times 8$, кадры с командами `LEFT/RIGHT/STRAIGHT` $\times 6$.

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Формирование последовательностей для GRU (реализовано в модуле `dataset.py`, приложение Д). Для обучения рекуррентного модуля датасет возвращает не одиночный кадр, а тензор `imgs_seq` размерности $(B, T = 4, 3, H, W)$. Для каждой строки `labels.csv` по колонке `cam` строится последовательность из T предыдущих кадров той же камеры; при нехватке истории ранний кадр дублируется. Данные лидара и скорость берутся только из текущего (последнего) кадра последовательности — это соответствует условиям инференса, где BEV-тензор и скорость доступны лишь для текущего тика.

Ускорение обучения. Применяется смешанная точность FP16 (`torch.cuda.amp: GradScaler + autocast`), что ускоряет обучение и снижает потребление видеопамати без потери качества. Оптимизатор — `AdamW` (`lr_backbone` = 3×10^{-5} , `lr_head` = 3×10^{-4} , `weight_decay` = 10^{-4}), планировщик — `CosineAnnealingWarmRestarts` (`T0` = 20, `T_mult` = 2, `eta_min` = 10^{-5}), ранняя остановка при `patience` = 20, максимум 80 эпох.

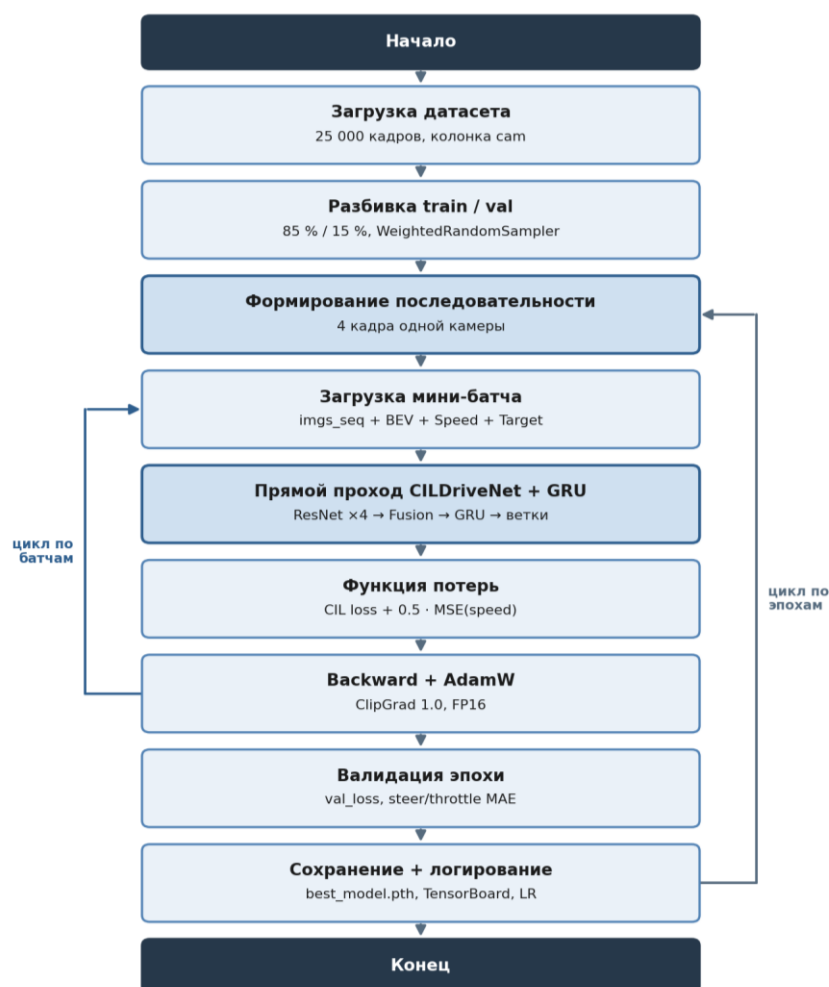


Рисунок 8 — Схема процесса обучения

Датасет разделяется на обучающую и валидационную выборки в соотношении 85:15, после чего батч поступает в модель, вычисляется branch-gated CIL-функция потерь, и оптимизатор AdamW обновляет веса. Метрики каждой эпохи (Train Loss, Val Loss, Val Steer MAE, Val Throttle MAE) логируются в TensorBoard.

Отформатировано: интервал после: 0 пт, междустрочный, одинарный

Отформатировано: интервал после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

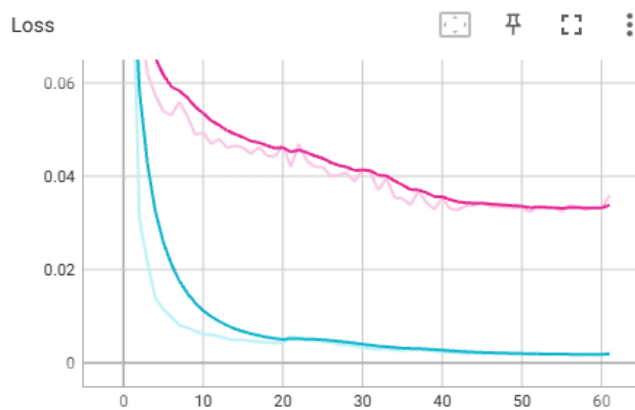
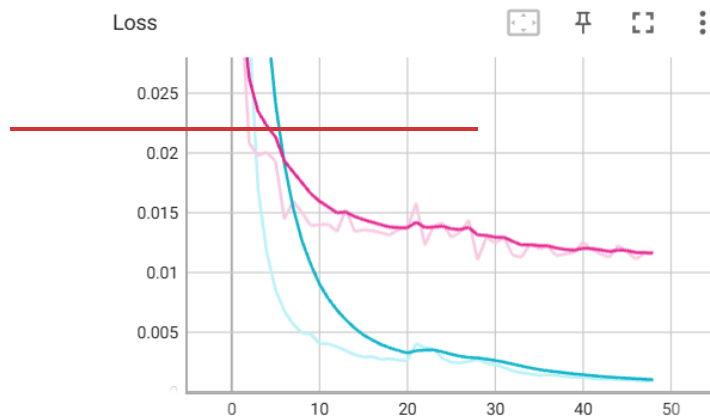


Рисунок 9 — Кривые обучения (TensorBoard): Loss train/val

По графикам на рисунке 9 видно, что обучающая и валидационная функции потерь монотонно убывают, расхождение между ними остаётся ограниченным, то есть существенного переобучения нет. Минимальное значение валидационной функции потерь достигается на 51-й эпохе, после чего наблюдается стабилизация показателей.

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, одинарный

Отформатировано: интервал после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

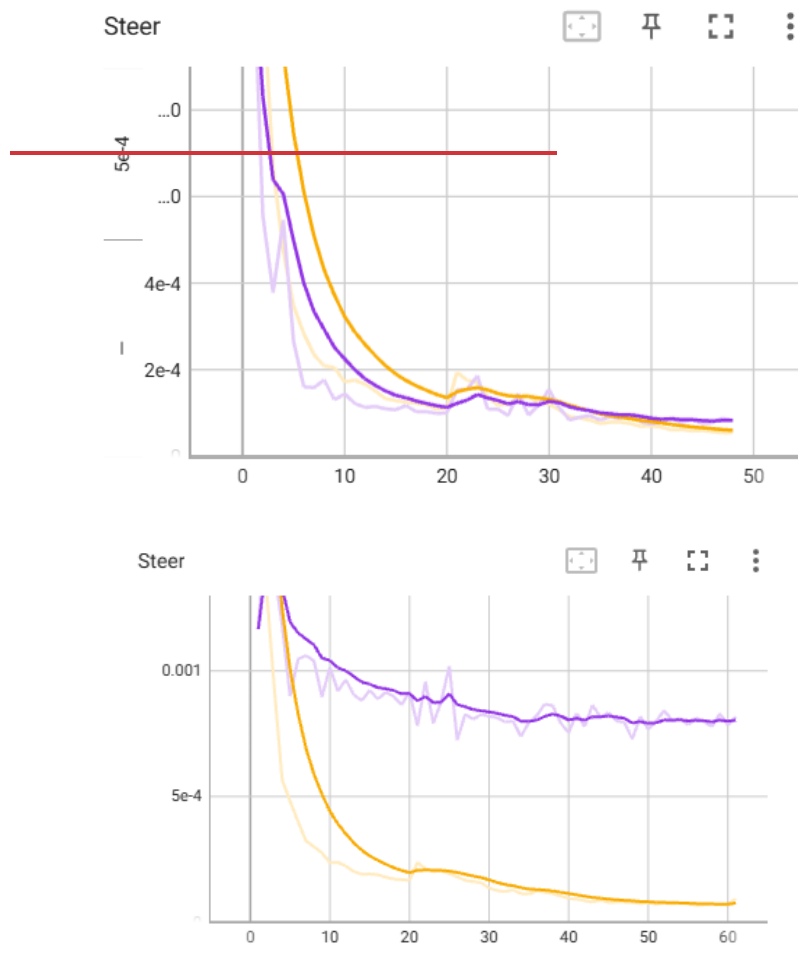


Рисунок 10 — Кривые ошибки по рулю

На рисунке 10 представлены кривые средней абсолютной ошибки (MAE) предсказания угла поворота рулевого колеса. Финальное значение Val Steer MAE = 0,0046 соответствует средней погрешности порядка 0,3° в физических единицах, что достаточно для удержания в полосе и выполнения поворотов.

отформатировано: английский (США), Без проверки правописания

Отформатировано: интервал после: 0 пт, междустрочный, 1,5 строки

Отформатировано: Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: По левому краю, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, междустрочный, 1,5 строки

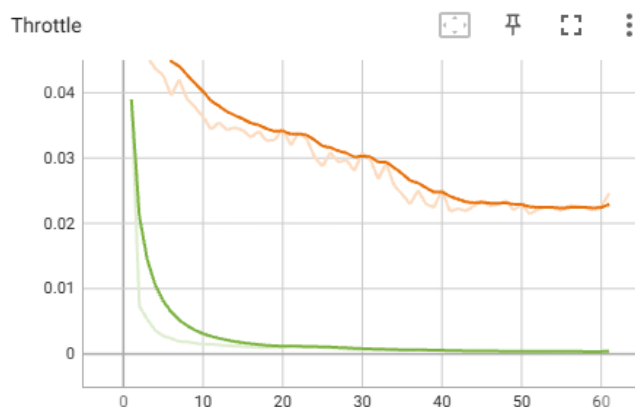
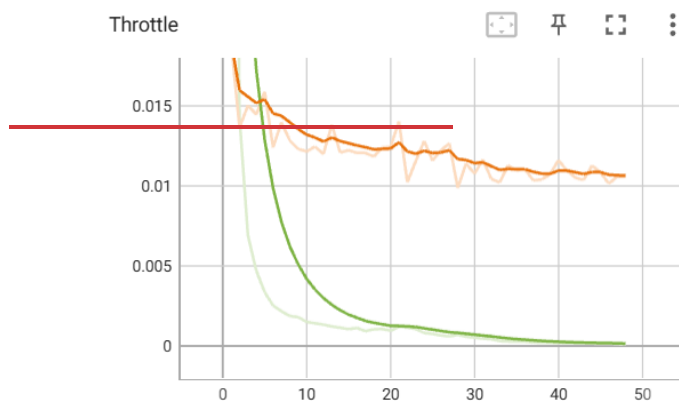


Рисунок 11 — Кривые ошибки по газу

На рисунке 11 показаны кривые ошибки предсказания значения throttle.

Поведение кривых сходно с кривыми по рулю, однако относительная ошибка несколько выше — это объясняется тем, что значение throttle вычисляется как прокси-метка из дельты скорости и содержит больше шума, чем геометрически определяемый угол руля.

Отформатировано: интервал после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

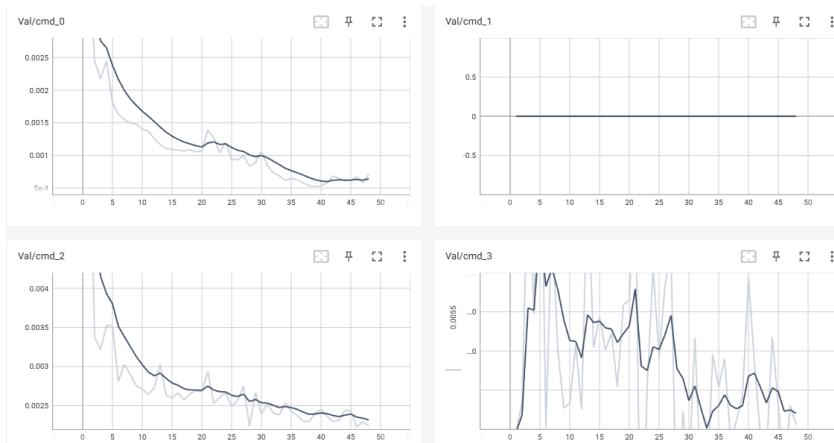


Рисунок 12 — Кривые ошибки по рулю и газу на валидации

Совмещённый график позволяет одновременно отслеживать обе компоненты ошибки и убедиться, что оптимизация одной не происходит за счёт ухудшения другой. Соотношение ошибок (Steer MAE $\approx$ 0,005, Throttle MAE $\approx$ 0,01) — рабочее для условного имитационного обучения.

Таблица 4 — Результаты обучения модели CILDriveNet

Модель	Лучшая эпоха	Train Loss	Val Loss	Val Steer MAE
Финальная модель (best_model.pth)	5128	0.001828	0.03230111	0.0046

5.8 Модуль инференса и безопасности (inference.py)

Скрипт inference.py реализует работу обученной модели в реальном времени в симуляторе (листинг — приложение Е). На каждом тике выполняется следующая последовательность:

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, одинарный

Отформатировано: интервал после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

отформатировано: английский (США)

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

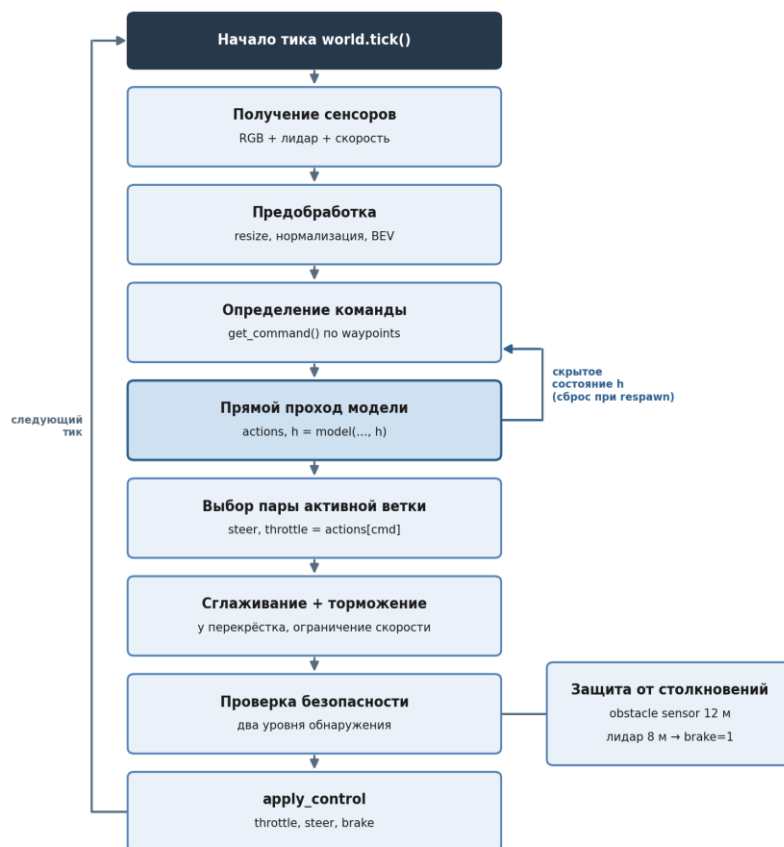


Рисунок 13 — Алгоритм работы модуля инференса

Алгоритм, изображённый на рисунке 13, выполняется на каждом тике симулятора и включает: получение сенсорных данных (RGB-кадр, облако точек лидара, скорость), их предобработку (resize, нормализация, формирование BEV-тензора), определение текущей навигационной команды по waypoints CARLA, прямой проход модели с передачей скрытого состояния GRU между тиками (длина последовательности $T = 1$), выбор пары (steer, throttle) из активной командной ветки, а также двухуровневую проверку

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

отформатировано: Шрифт: 12 пт, курсив, Цвет шрифта: Темно-красный

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 20 пт, после: 0 пт, междустрочный, 1,5 строки

безопасности — по сигналу `sensor.other.obstacle` и по содержимому ближней зоны лидарного BEV-тензора. При обнаружении препятствия управляющий сигнал `throttle` принудительно обнуляется, а `brake` устанавливается равным единице.

Скрытое состояние GRU `h` сохраняется между последовательными тиками симулятора, обеспечивая модель краткосрочной памяти о динамике движения. После аварии и автоматического `respawn` эго-машины скрытое состояние сбрасывается в ноль, поскольку история предшествующего эпизода более не релевантна. Дополнительно реализована двухуровневая защита от столкновений (`sensor.other.obstacle` на 12 м и проверка ближней зоны лидара на 8 м), обработка аварий с паузой 3 с и автоматическим `respawn`, торможение перед перекрёстком (`junction_ahead` на 28 м) и экспоненциальное сглаживание управляющих сигналов ($\text{steer} = 0,35 \cdot \text{prev} + 0,65 \cdot \text{raw}$; $\text{throttle} = 0,60 \cdot \text{prev} + 0,40 \cdot \text{raw}$).



Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

отформатировано: Шрифт: не курсив

Рисунок 14 — Интерфейс модуля инференса с HUD-оверлеем

На рисунке 14 показан пользовательский интерфейс скрипта инференса. В верхней части окна отображается HUD-оверлей с текущими значениями: скорость (км/ч), активная навигационная команда, предсказанные значения steer и throttle, а также ~~флаг наличия препятствия~~ количество аварий.



Рисунок 15 — ~~Флаг наличия препятствия "!! OBSTACLE !!"~~Выполнение поворота в режиме инференса

Отформатировано: интервал после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

отформатировано: русский

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Отформатировано: интервал после: 0 пт, междустрочный, 1,5 строки

6 Описание структур данных

6.1 Формат датасета

Датасет хранится в директории data/train/ и содержит три поддиректории:

- images/ — PNG-файлы 224×224 RGB (~250 КБ/кадр);
- lidar/ — NPY-файлы float32 5×200×200 (~780 КБ/кадр);
- labels.csv — метаданные всех кадров (см. таблицу 5).

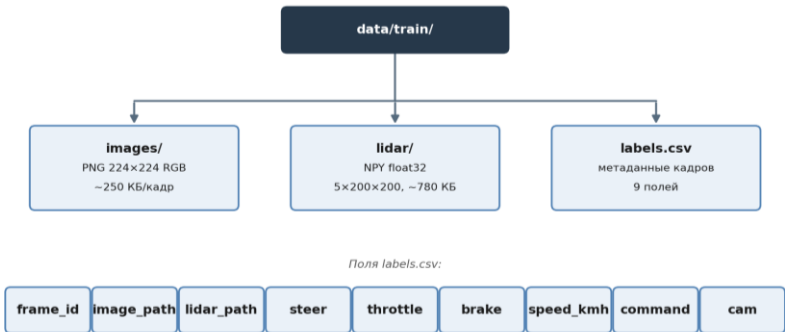


Рисунок 16 — Структура директории датасета

Файлам images/{frame_id}.png и lidar/{frame_id}.npy соответствует одна строка файла labels.csv с тем же значением frame_id, по этому frame_id сенсорные данные однозначно сопоставляются с меткой управления.

Таблица 5 — Поля файла labels.csv

Поле	Тип	Описание
frame_id	int	Порядковый номер кадра
image_path	str	Относительный путь к PNG
lidar_path	str	Относительный путь к NPY
steer	float32	Угол руля $\in [-1, 1]$ (модель Аккермана)
throttle	float32	Газ $\in [0, 1]$ (прокси по скорости)

Поле	Тип	Описание
brake	float32	Тормоз $\in [0, 1]$

Отформатировано: интервал Перед: 0 пт, после: 0 пт

Продолжение таблицы 5

Поле	Тип	Описание
speed_kmh	float32	Скорость км/ч (из дельты позиции)
command	int	0=FOLLOW, 1=LEFT, 2=RIGHT, 3=STRAIGHT
<u>cam</u>	<u>str</u>	<u>Камера: C=центр, L=лево, R=право (для построения последовательностей GRU)</u>

Отформатировано: интервал Перед: 0 пт, после: 0 пт

Отформатировано: интервал Перед: 0 пт, после: 0 пт

Отформатировано: интервал Перед: 0 пт, после: 0 пт

6.2 BEV-тензор лидара

отформатировано: Шрифт:

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

Облако точек лидара представляет собой неупорядоченный набор трёхмерных координат, неудобный для обработки свёрточной сетью. Поэтому функцией `lidar_to_bev()` оно преобразуется в регулярную сетку — представление вида сверху (Bird's Eye View, BEV). Пространство вокруг автомобиля в горизонтальной плоскости разбивается на сетку $BEV_SIZE \times BEV_SIZE$ ячеек, покрывающую область радиусом `LIDAR_RANGE` в каждом направлении; каждая точка облака попадает в соответствующую ячейку по координатам x и y .

Высота точек (координата z) кодируется отдельными каналами: диапазон $z \in [-3, 2]$ м делится на `HEIGHT_SLICES = 4` высотных среза, что позволяет различать объекты разной высоты — дорожное полотно, бордюры, корпуса автомобилей и верхние габариты. Пятый канал хранит нормализованную интенсивность отражения, несущую информацию о свойствах поверхности. Итоговый тензор размерности $5 \times 200 \times 200$ подаётся на вход свёрточного кодировщика `LidarEncoder`. Полный перечень параметров BEV-представления приведён в таблице

Таблица 6 — Параметры BEV-тензора лидара

<u>Параметр</u> <u>Параметр</u>	<u>Значение</u> <u>Значение</u>	<u>Описание</u> <u>Описание</u>
<u>LIDAR_RANGE</u> <u>LIDAR_RANGE</u>	<u>25.0 м</u> <u>25.0 м</u>	<u>Радиус покрытия</u> <u>Радиус</u> <u>покрытия в каждом</u> <u>направлении</u>
<u>BEV_SIZE</u> <u>BEV_SIZE</u>	<u>200×200</u> <u>200 пикс.</u>	<u>Размер сетки</u> <u>Размер сетки</u> <u>по каждой оси</u>
<u>BEV_RES</u> <u>BEV_RES</u>	<u>0.25 м/пкс</u> <u>0.25 м/пкс</u>	<u>Пространственное</u> <u>разрешение</u> <u>Пространствен</u> <u>ное разрешение</u>
<u>HEIGHT_SLICES</u> <u>HEIGHT_SLICES</u>	<u>44</u>	<u>Высотные каналы (z ∈</u> <u>[-3, 2] м)</u> <u>Число высотных</u> <u>каналов (z ∈ [-3, 2] м)</u>
<u>Канал 5</u> <u>Канал 5</u>	<u>Интенсивность</u> <u>Интенсивн</u> <u>ость</u>	<u>Нормализованная</u> <u>интенсивность</u> <u>отражения</u> <u>Нормализованн</u> <u>ая интенсивность</u> <u>отражения</u>
<u>Параметр</u>	<u>Значение</u>	<u>Описание</u>
<u>LIDAR_RANGE</u>	<u>25.0 м</u>	<u>Радиус покрытия в</u> <u>каждом</u> <u>направлении</u>

Отформатировано: интервал Перед: 20 пт, междустрочный, 1,5 строки

отформатировано: Шрифт: 12 пт

отформатировано: Шрифт: 12 пт

отформатировано: Шрифт: 12 пт

Отформатировано: По центру, интервал Перед: 20 пт

отформатировано: Шрифт: 12 пт

Отформатировано: По центру, интервал Перед: 20 пт

отформатировано: Шрифт: 12 пт

Отформатировано: По центру, интервал Перед: 20 пт

отформатировано: Шрифт: 12 пт

Отформатировано: интервал Перед: 20 пт

отформатировано: Шрифт: 12 пт

Отформатировано: интервал Перед: 20 пт

отформатировано: Шрифт: 12 пт

Отформатировано: интервал Перед: 20 пт

<u>Параметр</u> <u>Параметр</u>	<u>Значение</u> <u>Значение</u>	<u>Описание</u> <u>Описание</u>
<u>BEV_SIZE</u>	<u>200 пикс.</u>	<u>Размер сетки по каждой оси</u>
<u>BEV_RES</u>	<u>0.25 м/пкс</u>	<u>Пространственное разрешение</u>
<u>HEIGHT_SLICES</u>	<u>4</u>	<u>Число высотных каналов ($z \in [-3, 2]$ м)</u>
<u>Канал 5</u>	<u>Интенсивность</u>	<u>Нормализованная интенсивность отражения</u>

отформатировано: Шрифт: 12 пт

отформатировано: Шрифт: 12 пт

отформатировано: Шрифт: 12 пт

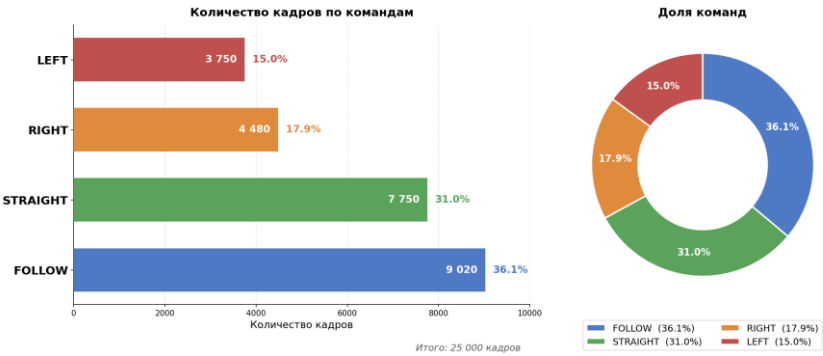
Отформатировано: По центру, интервал Перед: 20 пт

6.3 Метки навигационных команд

Функция `get_command()` (`utils.py`) анализирует waypoint впереди на 28 м. При обнаружении перекрёстка в радиусе 22 м: угловая разница $< -20^\circ \rightarrow$ LEFT; $> +20^\circ \rightarrow$ RIGHT; иначе $\rightarrow$ STRAIGHT. Без перекрёстка $\rightarrow$ FOLLOW. Промежуточные точки проверяются с шагом 7.3 м для надёжного обнаружения коротких junction-зон.

Отформатировано: По ширине, Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки, Поз.табуляции: 0,25 см, по левому краю

Отформатировано: междустрочный, 1,5 строки, Поз.табуляции: 0,25 см, по левому краю



Отформатировано: интервал после: 0 пт, междустрочный, 1,5 строки

Рисунок 17 — Распределение навигационных команд в датасете (Town10HD)

На рисунке 17 представлено распределение навигационных команд в собранном датасете (всего 25 000 кадров). Преобладает команда FOLLOW (36,1%), что соответствует естественному городскому трафику с большой долей прямолинейного движения. После дополнительного сбора поворотных сцен доли остальных команд составили: STRAIGHT — 31,0%, RIGHT — 17,9%, LEFT — 15,0%. Все поворотные команды уверенно превышают заложенный в требованиях минимум в 8%, что обеспечивает достаточное число примеров для обучения каждой командной ветки.

Отформатировано: интервал Перед: 0 пт, после: 0 пт, междустрочный, 1,5 строки

7 Тестирование модели в симуляционной среде

7.1 Методика тестирования

После завершения обучения работоспособность модели проверена в реальном времени в симуляторе CARLA 0.9.14 с помощью скрипта `inference_test.py` (листинг — приложение Ж). Тестирование проводилось в двух взаимодополняющих формах: количественной — по числовым показателям заездов, и качественной — по визуальному анализу поведения автомобиля в различных дорожных сценах.

Заезды выполнялись на нескольких картах. Карта `Town10HD` использовалась при сборе обучающих данных, тогда как карта `Town03` при обучении не использовалась. Прогон на незнакомых картах позволяет оценить не только качество вождения, но и обобщающую способность модели. В каждый заезд добавлялось до 30 управляемых `TrafficManager` автомобилей, что воспроизводит условия насыщенного городского движения.

На каждом шаге симуляции скрипт фиксирует управляющие сигналы и состояние машины, а датчик `sensor.other.collision` регистрирует столкновения. При аварии эго-машина автоматически перемещается в новую точку (`respawn`), и заезд продолжается без участия оператора. По завершении выводится сводная статистика, на основе которой заполнена таблица 8. Вспомогательные скрипты — очистки собранного датасета (`clean_dataset.py`, приложение И) и построения графиков обучения (`plot_results.py`, приложение К) — приведены в соответствующих приложениях.



СКРИНШОТ: окно тестового заезда с HUD — команда (CMD), угол руля, газ, скорость и счётчики столкновений (COLL) и respawn (RESP) /

Рисунок 18 — Интерфейс тестового заезда с HUD-оверлеем

7.2 Количественные результаты

Сводные показатели заездов по картам приведены в таблице 78. Для каждой карты фиксировались длительность заезда в кадрах, число столкновений, число успешно выполненных поворотов (LEFT и RIGHT) и число корректных остановок перед препятствием.

Таблица 78 — Результаты тестовых заездов модели по картам

Карта	Кадров	Ср. скорость, км/ч	Столкновений	Выполнено команд
Town10HD	2800	13,9	0	4 из 4
Town03	1400	15,9	0	4 из 4

Журнал заездов подтверждает корректную работу модели. На карте Town10HD за 2800 кадров модель отработала все четыре навигационные команды: следование по полосе (FOLLOW), проезд перекрёстка прямо (STRAIGHT), повороты налево (LEFT) и направо (RIGHT). Угол поворота руля закономерно мал на прямых участках ($|\text{steer}| < 0,05$) и возрастает в поворотах (например, при команде RIGHT $\text{steer} \approx +0,16$). Скорость удерживается в рабочем диапазоне 13–26 км/ч и снижается вплоть до полной остановки перед перекрёстком, что соответствует безопасному поведению. На карте Town03, не использовавшейся при обучении, за 1400 кадров модель также уверенно выполнила все команды без отклонений и без столкновений, что подтверждает её обобщающую способность.

7.3 Качественный анализ поведения

Визуальный анализ заездов показал, что модель уверенно воспроизводит основные паттерны городского вождения:

- удержание полосы движения на прямых участках;
- выполнение поворотов налево и направо в соответствии с навигационной командой;
- снижение скорости и остановка перед впереди идущим транспортом;
- торможение перед перекрёстком и его проезд согласно команде;
- избегание столкновений с другими участниками движения за счёт двухуровневой защиты.

[ВСТАВЬТЕ СКРИНШОТ: выполнение поворота (LEFT или RIGHT) на



перекрёстке]

Рисунок 19 — Выполнение поворота на перекрёстке в режиме инференса



[ВСТАВЬТЕ

СКРИНШОТ: остановка эго-машины перед впереди идущим автомобилем]

Рисунок 20 — Остановка перед впереди идущим транспортом

7.4 Обобщение на незнакомые карты

Особый интерес представляет поведение модели на карте Town03, не входившей в обучающую выборку. Несмотря на различия в геометрии дорог, расположении перекрёстков и визуальном оформлении сцен, модель

сохранила работоспособность и продолжила корректно выполнять повороты, торможения и удержание полосы. Это свидетельствует о том, что в процессе обучения модель выделила обобщённые признаки управления, а не запомнила конкретную карту.



ВСТАВЬТЕ СКРИНШОТ: тестовый заезд на карте Town03 (не использовалась при обучении)

Рисунок 21 — Тестовый заезд на незнакомой карте Town03

7.5 Статус разработки и практическая значимость

Разработанная модель представляет собой завершённый программный продукт — обученную нейросетевую систему автономной навигации, провалидированную в реалистичной симуляционной среде с трафиком. В рамках настоящей работы решена задача создания и проверки модели; перенос обученной модели на реальное транспортное средство (sim-to-real) относится к этапу промышленного внедрения и выходит за рамки работы.

Практическая значимость состоит в том, что создан функционирующий прототип системы автономной навигации, способный в реальном времени

отформатировано: не выделение цветом

управлять транспортным средством по навигационным командам. Полученные модель и программное обеспечение могут служить основой для дальнейших исследований и прикладных систем помощи водителю.

8 Анализ ограничений и некорректного поведения модели

Тестирование в симуляторе с трафиком выявило ряд ситуаций, в которых поведение модели отклоняется от желаемого. Анализ этих случаев важен для понимания границ применимости разработанного решения и определения направлений дальнейшей работы.

Выезд за пределы полосы. В части эпизодов модель смещала траекторию на смежную полосу, особенно на широких участках и при прохождении пологих поворотов. Основная причина — дисбаланс обучающей выборки: кадры с активным рулением встречаются значительно реже прямолинейного движения. После дополнительного сбора поворотных сцен доля команды LEFT доведена до 15%, что устранило её дефицит в выборке. Тем не менее остаточный перекос в сторону прямолинейного движения сохраняется, и взвешенная выборка (WeightedRandomSampler с коэффициентом $\times 8$ для поворотных кадров) компенсирует его лишь частично.

Столкновения с участниками движения. При плотном трафике модель не всегда своевременно снижала скорость перед впереди идущим транспортом. Поскольку обучающие данные собирались под управлением штатного автопилота, в них преобладает спокойный режим движения с малым числом экстренных торможений, и модель оказывается недостаточно обучена на краевых ситуациях. Частично проблема компенсируется двухуровневой аппаратной защитой, однако эта защита является внешней по отношению к модели и не заменяет корректного предсказания.

Резкость управляющих сигналов. Без сглаживания выход модели по рулю содержит высокочастотные колебания. В скрипте инференса применяется экспоненциальное сглаживание, однако оно вносит задержку реакции, что создаёт компромисс между плавностью и оперативностью управления.

Выявленные ограничения определяют дальнейшую работу: целенаправленный сбор сцен с торможением перед препятствием для лучшего воспроизведения экстренных манёвров; увеличение длины рекуррентной

последовательности для более раннего распознавания начала манёвра; введение в функцию потерь штрафа за резкое изменение руля между соседними кадрами; оценку обобщающей способности на карте Town03, не участвовавшей в обучении, для разделения ошибок переобучения и ошибок архитектуры. Наличие подобных ограничений закономерно для прототипа, обученного на ограниченной выборке, и не обесценивает основной результат: модель воспроизводит базовые паттерны вождения — удержание полосы, прохождение поворотов по команде и остановку перед препятствием.

ЗАКЛЮЧЕНИЕ

В ходе преддипломной практики была спроектирована, реализована и обучена мультимодальная нейросетевая модель CILDriveNet для задачи условного имитационного обучения автономному вождению в симуляторе CARLA 0.9.14.

Достигнутые результаты:

- проведён анализ существующих подходов; обоснован выбор метода CIL как наиболее подходящего для симуляционной среды с ограниченным бюджетом вычислений;

- спроектирована архитектура CILDriveNet: ResNet-18 (кодировщик изображений), лёгкая CNN (BEV-тензор лидара), MLP (скорость), блок слияния, рекуррентный модуль GRU (256 единиц, учёт истории из 4 кадров), четыре командные ветки и вспомогательная голова предсказания скорости (Speed Head)~~четыре командные ветки;~~

- реализован конвейер сбора ~~2520~~ 000 обучающих кадров на карте Town10HD; решена техническая проблема CARLA 0.9.14 API (нулевые значения get_control/get_velocity) — применён обходной путь на основе дельты позиции и модели Аккермана;

- добавлен рекуррентный модуль GRU: модель учитывает историю из 4 последовательных кадров, при инференсе скрытое состояние передаётся между тиками симулятора и сбрасывается после аварии;

- модель успешно обучена; реализована двухуровневая защита от столкновений (sensor.other.obstacle + LIDAR-based check);

- проведено тестирование модели в реальном времени в симуляторе CARLA на нескольких картах (Town10HD, Town03) с трафиком; модель уверенно выполняет повороты, торможение и удержание полосы, в том числе на картах, не входивших в обучение.

Текущие результаты обучения:

Epoch 05128 | Train 0.001828 Val 0.0323411 | S 0.0001 T 0.0099 | L 0.0000 R 0.0022 Str 0.0046 | 24375.5s

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

Отформатировано: Отступ: Слева: 0 см, Первая строка: 1,25 см, междустрочный, 1,5 строки

Отформатировано: Отступ: Слева: 0 см, Первая строка: 1,25 см, междустрочный, 1,5 строки

Отформатировано: Отступ: Первая строка: 1,25 см, интервал Перед: 0 пт

Дальнейшие шаги в рамках дипломной работы:
количественная оценка на независимом тестовом маршруте,
сравнение с базовой линией CHRS,
возможно расширение датасета за счёт сбора данных на дополнительных картах
(Town01, Town03).

← **Отформатировано:** интервал Перед: 0 пт,
междустрочный, 1,5 строки

← **Отформатировано:** междустрочный, 1,5 строки

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Codevilla F., Müller M., López A. и др. End-to-end Driving via Conditional Imitation Learning // IEEE International Conference on Robotics and Automation (ICRA). — Brisbane: IEEE, 2018. — С. 4693–4700. — URL: <https://doi.org/10.1109/ICRA.2018.8460487>
2. Codevilla F., Santana E., López A.M., Gaidon A. Exploring the Limitations of Behavior Cloning for Autonomous Driving // IEEE/CVF International Conference on Computer Vision (ICCV). — Seoul: IEEE, 2019. — С. 9329–9338. — URL: <https://doi.org/10.1109/ICCV.2019.00942>
3. Chen D., Zhou B., Koltun V., Krähenbühl P. Learning by Cheating // Proceedings of the Conference on Robot Learning (CoRL 2019). — PMLR, 2020. — Т. 100. — С. 66–75. — URL: <https://proceedings.mlr.press/v100/chen20a.html>
4. Zhang Z., Liniger A., Dai D. и др. End-to-End Urban Driving by Imitating a Reinforcement Learning Coach (Roach) // IEEE/CVF International Conference on Computer Vision (ICCV). — Montreal: IEEE, 2021. — С. 15222–15232. — URL: <https://doi.org/10.1109/ICCV48922.2021.01494>
5. Dosovitskiy A., Ros G., Codevilla F. и др. CARLA: An Open Urban Driving Simulator // Proceedings of the 1st Annual Conference on Robot Learning (CoRL 2017). — PMLR, 2017. — Т. 78. — С. 1–16. — Текст: электронный. — URL: <https://proceedings.mlr.press/v78/dosovitskiy17a.html> (дата обращения: 10.05.2026).
6. Prakash A., Chitta K., Geiger A. Multi-Modal Fusion Transformer for End-to-End Autonomous Driving (TransFuser) // IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). — Nashville: IEEE, 2021. — С. 7077–7087. — URL: <https://doi.org/10.1109/CVPR46437.2021.00700>
7. He K., Zhang X., Ren S., Sun J. Deep Residual Learning for Image Recognition // IEEE Conference on Computer Vision and Pattern Recognition (CVPR). — Las Vegas: IEEE, 2016. — С. 770–778. — URL: <https://doi.org/10.1109/CVPR.2016.90>

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

8. LeCun Y., Bengio Y., Hinton G. Deep Learning // Nature. — 2015. — Т. 521, № 7553. — С. 436–444. — URL: <https://doi.org/10.1038/nature14539>

9. Николенко С.И., Кадуринов А.А., Архангельская Е.О. Глубокое обучение. Погружение в мир нейронных сетей. — СПб.: Питер, 2018. — 480 с. — (Серия «Библиотека программиста»). — ISBN 978-5-496-02536-2.

10. Гудфеллоу Я., Бенджио И., Курвилль А. Глубокое обучение / пер. с англ. А.А. Слинкина. — 2-е изд., испр. — М.: ДМК Пресс, 2018. — 652 с. — ISBN 978-5-97060-618-6.

11. Шолле Ф. Глубокое обучение на Python / пер. с англ. А. Киселева. — СПб.: Питер, 2018. — 400 с. — (Серия «Библиотека программиста»). — ISBN 978-5-4461-0770-4.

12. Саттон Р.С., Барто Э.Дж. Обучение с подкреплением: введение / пер. с англ. А.А. Слинкина. — 2-е изд. — М.: ДМК Пресс, 2020. — 552 с. — ISBN 978-5-97060-097-9.

13. Хайкин С. Нейронные сети: полный курс / пер. с англ. Н.Н. Куссуль, А.Ю. Шелестова. — 2-е изд., испр. — М.: Издательский дом «Вильямс», 2006. — 1104 с. — ISBN 5-8459-0890-6.

14. Гонсалес Р., Вудс Р. Цифровая обработка изображений / пер. с англ. Л.И. Рубанова, П.А. Чочиа; науч. ред. перевода П.А. Чочиа. — 3-е изд., испр. и доп. — М.: Техносфера, 2012. — 1104 с. — ISBN 978-5-94836-331-8.

15. Ярышев С.Н., Рыжова В.А., Волынский М.А. Системы технического зрения в задачах обнаружения, измерения, управления: учебное пособие. — СПб.: Университет ИТМО, 2023. — 110 с.

16. Станкевич Л.А. Когнитивные системы и роботы: монография / Министерство науки и высшего образования Российской Федерации, Санкт-Петербургский политехнический университет Петра Великого. — СПб.: Политех-Пресс, 2019. — 630 с. — ISBN 978-5-7422-6610-6.

17. Лозовецкий В.В. Беспилотные транспортные средства. Инновационные роботизированные системы на суше, воде и воздухе: учебное

пособие для среднего профессионального образования. — СПб.: Лань, 2022. — 364 с. — (Среднее профессиональное образование). — ISBN 978-5-8114-9270-7.

18. Воронцов К.В. Математические методы обучения по прецедентам (теория обучения машин): курс лекций. — М.: МФТИ, Школа анализа данных Яндекса, 2024. — 141 с.

19. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. — М.: Стандартинформ, 2017. — 27 с. — URL: https://guap.ru/standards/db/docs/gost_7.32-2017.pdf

20. ГОСТ Р 7.0.100-2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — М.: Стандартинформ, 2018. — 124 с. — URL: https://guap.ru/standards/db/docs/gost_r_7.0.100-2018.pdf

21. [Choi K., van Merriënboer B., Gulcehre C. и др. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation // Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing \(EMNLP\). — Doha: Association for Computational Linguistics, 2014. — С. 1724–1734. — URL: <https://doi.org/10.3115/v1/D14-1179>](#)

отформатировано: Основной шрифт абзаца, Цвет шрифта: Авто, английский (США)

отформатировано: Основной шрифт абзаца, Цвет шрифта: Авто

отформатировано: Основной шрифт абзаца, Цвет шрифта: Авто, английский (США)

отформатировано: Основной шрифт абзаца, Цвет шрифта: Авто

отформатировано: Основной шрифт абзаца, Цвет шрифта: Авто, английский (США)

Отформатировано: интервал Перед: 0 пт, междустрочный, 1,5 строки

отформатировано: Основной шрифт абзаца, Цвет шрифта: Авто

отформатировано: Основной шрифт абзаца, Цвет шрифта: Авто, английский (США)

отформатировано: подчеркивание, Цвет шрифта: Синий, английский (США)

отформатировано: английский (США)

ПРИЛОЖЕНИЕ А

Листинг модуля model.py (архитектура CILDriveNet с GRU)

```
"""
CILDriveNet — мультимодальная CIL-модель с GRU для временной памяти.

Архитектура:
Camera (ResNet-18, ImageNet pretrained) -> 512-d
LiDAR (CNN, 5x200x200 BEV) -> 128-d
Speed (MLP, 1->64) -> 64-d
Fusion FC(704->256)
GRU (256->256) <- запоминает историю T кадров
Branches x4: FC(256->128->2) [steer=tanh, throttle=sigmoid]

При обучении: imgs_seq (B, T, 3, H, W) — T последовательных кадров.
При инференсе: imgs_seq (B, 1, 3, H, W) или (B, 3, H, W), h передаётся между тиками.
"""

import torch
import torch.nn as nn

try:
    from torchvision.models import resnet18, ResNet18_Weights
    def _resnet18():
        return resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
except (ImportError, AttributeError):
    from torchvision import models
    def _resnet18():
        return models.resnet18(pretrained=True)

# — Sub-modules —

class LidarEncoder(nn.Module):
    """Tiny CNN: (5, 200, 200) BEV -> 128-d vector."""
    def __init__(self, in_ch: int = 5, out_dim: int = 128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_ch, 32, 3, stride=2, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(inplace=True),
            nn.Conv2d(32, 64, 3, stride=2, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(inplace=True),
            nn.Conv2d(64, out_dim, 3, stride=2, padding=1),
            nn.BatchNorm2d(out_dim), nn.ReLU(inplace=True),
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
        )
    def forward(self, x): return self.net(x)

class SpeedEncoder(nn.Module):
    """MLP: scalar speed [0,1] -> 64-d vector."""
    def __init__(self, out_dim: int = 64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(1, out_dim), nn.ReLU(inplace=True),
            nn.Linear(out_dim, out_dim), nn.ReLU(inplace=True),
        )
    def forward(self, x): return self.net(x)

# — Main model —

class CILDriveNet(nn.Module):
    """
    Conditional Imitation Learning + GRU temporal memory.

    Inputs:
    imgs_seq (B, T, 3, 224, 224) -- T sequential camera frames
    lidar_bev (B, 5, 200, 200) -- LiDAR BEV of current frame
    speed (B, 1) -- normalised speed [0,1]
    h (1, B, 256) or None -- GRU hidden state (None = zeros)

    Outputs:
    actions (B, 4, 2) -- steer (tanh) + throttle (sigmoid) per command
    """
```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

        speed_pred (B, 1)      -- predicted speed (auxiliary loss, sigmoid)
        h_new       (1, B, 256) -- updated GRU hidden state (keep between ticks)

Command mapping:  0=FOLLOW  1=LEFT  2=RIGHT  3=STRAIGHT
"""

NUM_COMMANDS = 4
CAM_DIM       = 512
LIDAR_DIM     = 128
SPEED_DIM     = 64
FUSE_DIM      = CAM_DIM + LIDAR_DIM + SPEED_DIM    # 704
GRU_DIM       = 256

def __init__(self):
    super().__init__()

    resnet = _resnet18()
    self.camera_encoder = nn.Sequential(*list(resnet.children())[:-1])

    self.lidar_encoder = LidarEncoder(in_ch=5, out_dim=self.LIDAR_DIM)
    self.speed_encoder = SpeedEncoder(out_dim=self.SPEED_DIM)

    self.fusion = nn.Sequential(
        nn.Linear(self.FUSE_DIM, 512), nn.ReLU(inplace=True),
        nn.Linear(512, self.GRU_DIM),  nn.ReLU(inplace=True),
    )

    # GRU: обрабатывает последовательность fusion-векторов
    self.gru = nn.GRU(
        input_size=self.GRU_DIM,
        hidden_size=self.GRU_DIM,
        num_layers=1,
        batch_first=False,    # (T, B, GRU_DIM)
    )
    self.gru_dropout = nn.Dropout(p=0.3)

    self.branches = nn.ModuleList([
        nn.Sequential(
            nn.Linear(self.GRU_DIM, 128), nn.ReLU(inplace=True),
            nn.Linear(128, 2),
        )
        for _ in range(self.NUM_COMMANDS)
    ])

    # Вспомогательная голова предсказания скорости из визуальных признаков
    self.speed_head = nn.Sequential(
        nn.Linear(self.CAM_DIM + self.LIDAR_DIM, 128),
        nn.ReLU(inplace=True),
        nn.Linear(128, 1),
        nn.Sigmoid(),
    )

def forward(self, imgs_seq, lidar_bev, speed, h=None):
    # Поддержка одиночного кадра (B, 3, H, W) -> (B, 1, 3, H, W)
    if imgs_seq.dim() == 4:
        imgs_seq = imgs_seq.unsqueeze(1)

    B, T, C, H, W = imgs_seq.shape

    # — Закодировать все T кадров камерой —————
    imgs_flat = imgs_seq.reshape(B * T, C, H, W)
    cam_f_all = self.camera_encoder(imgs_flat).flatten(1)    # (B*T, 512)

    # — LiDAR и скорость — текущий кадр, растягиваем на T —————
    lidar_f = self.lidar_encoder(lidar_bev)    # (B, 128)
    speed_f = self.speed_encoder(speed)        # (B, 64)

    lidar_f_all = lidar_f.unsqueeze(1).expand(-1, T, -1).reshape(B * T, -1)
    speed_f_all = speed_f.unsqueeze(1).expand(-1, T, -1).reshape(B * T, -1)

    # — Aux: предсказать скорость из текущего кадра —————
    cam_f_cur = cam_f_all.view(B, T, -1)[: , -1, :]    # (B, 512)
    speed_pred = self.speed_head(torch.cat([cam_f_cur, lidar_f], dim=1))

    # — Fusion: конкатенация → FC —————
    fused = torch.cat([cam_f_all, lidar_f_all, speed_f_all], dim=1)    # (B*T, 704)
    fused = self.fusion(fused)                                          # (B*T, 256)

    # — GRU —————
    fused = fused.view(B, T, self.GRU_DIM).permute(1, 0, 2)    # (T, B, 256)

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

gru_out, h_new = self.gru(fused, h) # (T,B,256), (1,B,256)
x = self.gru_dropout(gru_out[-1]) # (B, 256)

# — Командные ветки
branch_outs = []
for branch in self.branches:
    raw = branch(x)
    steer = torch.tanh(raw[:, 0:1])
    throttle = torch.sigmoid(raw[:, 1:2])
    branch_outs.append(torch.cat([steer, throttle], dim=1))

actions = torch.stack(branch_outs, dim=1) # (B, 4, 2)
return actions, speed_pred, h_new

```

отформатировано: английский (США)

отформатировано: английский (США)

ПРИЛОЖЕНИЕ Б

Листинг модуля `utils.py` (BEV-проекция, навигационные команды, проверка препятствий)

```
"""
Shared utilities for LIDAR BEV projection and navigation command inference.
Used in both data collection and inference so behaviour stays identical.
"""
import numpy as np

# --- LIDAR BEV parameters ---
LIDAR_RANGE = 25.0 # metres in each direction from vehicle centre
BEV_SIZE = 200 # pixels (200x200 grid)
BEV_RES = (LIDAR_RANGE * 2) / BEV_SIZE # 0.25 m / pixel
HEIGHT_SLICES = 4 # vertical bins; +1 intensity channel → 5 total channels
Z_RANGE = (-3.0, 2.0)

# --- Command identifiers ---
CMD_FOLLOW = 0
CMD_LEFT = 1
CMD_RIGHT = 2
CMD_STRAIGHT = 3
CMD_NAMES = {CMD_FOLLOW: "FOLLOW", CMD_LEFT: "LEFT",
              CMD_RIGHT: "RIGHT", CMD_STRAIGHT: "STRAIGHT"}

# --- ImageNet normalisation (used with pretrained ResNet-18) ---
IMG_MEAN = np.array([0.485, 0.456, 0.406], dtype=np.float32)
IMG_STD = np.array([0.229, 0.224, 0.225], dtype=np.float32)
IMG_SIZE = (224, 224)

MAX_SPEED_KMH = 60.0 # normalisation factor

def lidar_to_bev(pts: np.ndarray) -> np.ndarray:
    """
    Convert a LIDAR point cloud (N, 4) [x, y, z, intensity] to a
    bird's-eye-view tensor of shape (HEIGHT_SLICES+1, BEV_SIZE, BEV_SIZE).

    CARLA sensor frame: x=forward, y=right, z=up.
    BEV axes: axis-0 (row) = y (right-down), axis-1 (col) = x (forward-right).
    """
    x_lo, x_hi = -LIDAR_RANGE, LIDAR_RANGE
    y_lo, y_hi = -LIDAR_RANGE, LIDAR_RANGE
    z_lo, z_hi = Z_RANGE

    bev = np.zeros((HEIGHT_SLICES + 1, BEV_SIZE, BEV_SIZE), dtype=np.float32)

    if len(pts) == 0:
        return bev

    mask = (
        (pts[:, 0] > x_lo) & (pts[:, 0] < x_hi) &
        (pts[:, 1] > y_lo) & (pts[:, 1] < y_hi) &
        (pts[:, 2] > z_lo) & (pts[:, 2] < z_hi)
    )
    pts = pts[mask]
    if len(pts) == 0:
        return bev

    col = np.floor((pts[:, 0] - x_lo) / BEV_RES).astype(np.int32)
    row = np.floor((pts[:, 1] - y_lo) / BEV_RES).astype(np.int32)
    col = np.clip(col, 0, BEV_SIZE - 1)
    row = np.clip(row, 0, BEV_SIZE - 1)

    z_step = (z_hi - z_lo) / HEIGHT_SLICES
    for k in range(HEIGHT_SLICES):
        z_bot = z_lo + k * z_step
        z_top = z_lo + (k + 1) * z_step
        ch = (pts[:, 2] >= z_bot) & (pts[:, 2] < z_top)
        np.maximum.at(bev[k], (row[ch], col[ch]), 1.0)

    # Intensity channel (normalised density)
    np.add.at(bev[HEIGHT_SLICES], (row, col), pts[:, 3])
    vmax = bev[HEIGHT_SLICES].max()
```

отформатировано: английский (США)

```

if vmax > 0:
    bev[HEIGHT_SLICES] /= vmax

return bev

def get_command(vehicle, world_map,
               lookahead_m: float = 28.0,
               junction_ahead_m: float = 22.0) -> int:
    """
    Infer the upcoming high-level navigation command.

    lookahead_m      - смотрим на waypoint ВПЕРЕДИ на это расстояние для определения
                        yaw-разницы. Должно быть > junction_ahead_m, чтобы точка
                        оказалась уже за перекрёстком и давала правильное направление.
    junction_ahead_m - считать «рядом с перекрёстком» если он ближе этого расстояния.
                        22 м при 30 км/ч → ~2.6 с реакции для поворота.
    """
    import carla

    loc = vehicle.get_location()
    wp = world_map.get_waypoint(
        loc, project_to_road=True, lane_type=carla.LaneType.Driving
    )
    if wp is None:
        return CMD_FOLLOW

    vehicle_yaw = vehicle.get_transform().rotation.yaw
    next_wps = wp.next(lookahead_m)
    if not next_wps:
        return CMD_FOLLOW

    ahead_wp = next_wps[0]
    diff = ((ahead_wp.transform.rotation.yaw - vehicle_yaw) + 180) % 360 - 180

    # Перекрёсток: текущий WP внутри junction ИЛИ junction в пределах junction ahead m.
    # Проверяем несколько точек вдоль дороги – wp.next(d) может «проскочить» через
    # короткую junction-зону если d велико.
    near_junction = wp.is_junction
    if not near_junction:
        step = junction_ahead_m / 3.0
        cur = wp
        for _ in range(3):
            nxt = cur.next(step)
            if not nxt:
                break
            cur = nxt[0]
            if cur.is_junction:
                near_junction = True
                break

    if near_junction:
        if diff < -20:
            return CMD_LEFT
        elif diff > 20:
            return CMD_RIGHT
        else:
            return CMD_STRAIGHT

    return CMD_FOLLOW

def obstacle_ahead(pts: np.ndarray,
                  max_dist: float = 8.0,
                  half_width: float = 1.5,
                  min_z: float = 0.2,
                  max_z: float = 2.2,
                  min_pts: int = 8) -> bool:
    """Safety check: returns True if >= min_pts LIDAR points are found
    directly ahead within max_dist metres and within the lane width."""
    if len(pts) == 0:
        return False
    mask = (
        (pts[:, 0] > 1.0) & (pts[:, 0] < max_dist) &
        (np.abs(pts[:, 1]) < half_width) &
        (pts[:, 2] > min_z) & (pts[:, 2] < max_z)
    )
    return int(mask.sum()) >= min_pts

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

ПРИЛОЖЕНИЕ В

Листинг модуля collect_data.py (сбор обучающих данных)

```
"""
Сбор данных для CIL-модели — три камеры + LiDAR.

Три камеры (центр, левая -0.5м, правая +0.5м) на каждый тик дают три строки в CSV
с разными значениями steer. Это решает главную проблему обучения — covariate shift:
модель получает примеры того, как выглядит дорога при смещении от центра полосы
и какой руль нужен чтобы вернуться.

Левая камера (y=-0.5м): steer = original + CORRECTION (+0.10)
                                Дорога смещена вправо → нужно подруливать вправо
Центральная (y= 0.0м): steer = original
Правая камера (y=+0.5м): steer = original - CORRECTION (-0.10)
                                Дорога смещена влево → нужно подруливать влево

LiDAR один (центральный) — одна .npy-запись на тик, на неё ссылаются все три строки CSV.

На каждый тик сохраняется:
• 3 PNG-кадра (images/)
• 1 NPY-файл лидара (lidar/)
• 3 строки в labels.csv

При TOTAL_FRAMES=24000 нужно ~8000 тиков реальной езды (~7 минут).
"""

import os, sys, csv, time, queue, random
import numpy as np
import cv2
from tqdm import tqdm

# — Конфиг —————

TOWN = "Town10HD"
TOTAL_FRAMES = 24_000 # итоговых строк в CSV (÷3 = тиков езды)
IMAGE_SIZE = (640, 480)
SAVE_SIZE = (224, 224)
FPS = 20
DELTA_SEC = 1.0 / FPS
NPC_COUNT = 25
WEATHER_INTERVAL = 600 # тиков между сменой погоды
TM_PORT = 8000

TARGET_SPEED = 30.0 # км/ч
WHEELBASE = 2.875 # м, Tesla Model 3
MAX_STEER_RAD = np.deg2rad(70.0)

WARMUP_TICKS = 200
MIN_SPEED_KMH = 1.0
STUCK_KMH = 1.5
STUCK_TICKS_MAX = 300 # 15 сек → перезапуск автопилота (без телепорта)

STEER_CORRECTION = 0.10 # коррекция руля для боковых камер

# (название, смещение y в м, коррекция steer)
# y<0 = влево, y>0 = вправо (система координат CARLA: y=right)
CAM_CONFIGS = [
    ('C', 0.0, 0.00), # центральная камера — без коррекции
    ('L', -0.5, +STEER_CORRECTION), # левая — нужно подруливать вправо
    ('R', +0.5, -STEER_CORRECTION), # правая — нужно подруливать влево
]

FRESH_START = True

OUTPUT_DIR = os.path.join(os.path.dirname(__file__), "..", "data", "train")
IMAGES_DIR = os.path.join(OUTPUT_DIR, "images")
LIDAR_DIR = os.path.join(OUTPUT_DIR, "lidar")
CSV_PATH = os.path.join(OUTPUT_DIR, "labels.csv")

if FRESH_START:
    import shutil
    for _d in (IMAGES_DIR, LIDAR_DIR):
        if os.path.exists(_d):
            shutil.rmtree(_d)
    if os.path.exists(CSV_PATH):
        os.remove(CSV_PATH)
```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

    print("    Старый датасет удалён. (FRESH_START=True).")

os.makedirs(IMAGES_DIR, exist_ok=True)
os.makedirs(LIDAR_DIR, exist_ok=True)

sys.path.insert(0, os.path.dirname(__file__))
from utils import lidar_to_bev, get_command, CMD_NAMES, HEIGHT_SLICES, BEV_SIZE

# — Delta-based управление
def compute_controls(loc_before, yaw_before, loc_after, yaw_after):
    dx, dy = loc_after.x - loc_before.x, loc_after.y - loc_before.y
    speed_ms = np.sqrt(dx**2 + dy**2) / DELTA_SEC
    speed_kmh = float(speed_ms * 3.6)

    yaw_diff = ((yaw_after - yaw_before) + 180) % 360 - 180
    if speed_kmh > 1.0:
        yaw_rate = (yaw_diff * np.pi / 180.0) / DELTA_SEC
        delta = np.arctan(yaw_rate * WHEELBASE / speed_ms)
        steer = float(np.clip(delta / MAX_STEER_RAD, -1.0, 1.0))
    else:
        steer = 0.0

    if speed_kmh < TARGET_SPEED * 0.85:
        throttle, brake = 0.70, 0.0
    elif speed_kmh > TARGET_SPEED * 1.10:
        throttle, brake = 0.0, 0.25
    else:
        throttle, brake = 0.45, 0.0

    return speed_kmh, steer, throttle, brake

# — Порода
def apply_weather(world):
    import carla
    world.set_weather(carla.WeatherParameters(
        cloudiness = np.random.uniform(5, 70),
        precipitation = np.random.uniform(0, 20),
        wind_intensity = np.random.uniform(5, 40),
        sun_azimuth_angle = np.random.uniform(0, 360),
        sun_altitude_angle = np.random.uniform(35, 75),
        fog_density = np.random.uniform(0, 5),
        wetness = np.random.uniform(0, 25),
    ))

# — NPC
def spawn_npcs(world, tm, count, spawn_points, bp_lib):
    npc_bps = bp_lib.filter('vehicle.*')
    npcs, used = [], set()
    for _ in range(count * 5):
        if len(npcs) >= count: break
        sp = random.choice(spawn_points)
        key = (round(sp.location.x, 1), round(sp.location.y, 1))
        if key in used: continue
        used.add(key)
        bp = random.choice(npc_bps)
        if bp.has_attribute('color'):
            bp.set_attribute('color', random.choice(
                bp.get_attribute('color').recommended_values))
        npc = world.try_spawn_actor(bp, sp)
        if npc:
            npc.set_autopilot(True, TM_PORT)
            npcs.append(npc)
    return npcs

# — Main
def main():
    import carla

    np.random.seed(42); random.seed(42)

    print(f"Загрузка {TOWN} ...")

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)


```

client = carla.Client('127.0.0.1', 2000)
client.set_timeout(20.0)
world = client.get_world()

current = world.get_map().name.split('/')[ -1]
if current != TOWN:
    print(f" {current} → {TOWN} ...")
    client.load_world(TOWN); time.sleep(4.0)
    world = client.get_world()
world_map = world.get_map()

# — Синхронный режим —
settings = world.get_settings()
settings.synchronous_mode = True
settings.fixed_delta_seconds = DELTA_SEC
world.apply_settings(settings)

# — TrafficManager —
tm = client.get_trafficmanager(TM_PORT)
tm.set_synchronous_mode(True)
tm.set_global_distance_to_leading_vehicle(2.5)
tm.set_hybrid_physics_mode(False)
try: tm.set_respawn_dormant_vehicles(True)
except AttributeError: pass

# — Эго-машина —
bp_lib = world.get_blueprint_library()
vehicle_bp = bp_lib.find('vehicle.tesla.model3')
spawn_points = world_map.get_spawn_points()

vehicle = None
for sp in random.sample(spawn_points, min(20, len(spawn_points))):
    vehicle = world.try_spawn_actor(vehicle_bp, sp)
    if vehicle: break
if not vehicle:
    print("❌ Не удалось заspawnить машину."); return

vehicle.set_autopilot(True, TM_PORT)
tm.vehicle_percentage_speed_difference(vehicle, -50)
print(f"✅ Эго-машина_id={vehicle.id}")

# — NPC —
print(f"⚠️ Spawn {NPC_COUNT} NPC ...")
npcs = spawn_npcs(world, tm, NPC_COUNT, spawn_points, bp_lib)
print(f"✅ NPC: {len(npcs)}")

# — Три камеры —
cam_actors = {}
cam_queues = {}
for name, y_offset, _ in CAM_CONFIGS:
    cam_bp = bp_lib.find('sensor.camera.rgb')
    cam_bp.set_attribute('image_size_x', str(IMAGE_SIZE[0]))
    cam_bp.set_attribute('image_size_y', str(IMAGE_SIZE[1]))
    cam_bp.set_attribute('fov', '90')
    cam_bp.set_attribute('enable_postprocess_effects', 'False')
    cam = world.spawn_actor(
        cam_bp,
        carla.Transform(carla.Location(x=2.5, y=y_offset, z=1.0),
            carla.Rotation(yaw=0)),
        attach_to=vehicle)
    q = queue.Queue(maxsize=5)
    cam.listen(lambda img, q=q: q.put(img) if not q.full() else None)
    cam_actors[name] = cam
    cam_queues[name] = q
print(f"✅ Камеры: {list(cam_actors.keys())} (центр / левая / правая)")

# — LiDAR (один, центральный) —
lidar_bp = bp_lib.find('sensor.lidar.ray_cast')
lidar_bp.set_attribute('channels', '32')
lidar_bp.set_attribute('range', '40')
lidar_bp.set_attribute('points_per_second', '56000')
lidar_bp.set_attribute('rotation_frequency', str(FPS))
lidar_bp.set_attribute('upper_fov', '10.0')
lidar_bp.set_attribute('lower_fov', '-30.0')
lidar_sensor = world.spawn_actor(
    lidar_bp,
    carla.Transform(carla.Location(x=0.0, z=2.4)),
    attach_to=vehicle)
lidar_queue = queue.Queue(maxsize=5)

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

lidar_sensor.listen(lambda d: lidar_queue.put(d) if not lidar_queue.full() else None)

# — Начальная порада —————
apply_weather(world)
weather_tick = 0

# — Прогрев —————
print(f"Прогрев ({WARMUP_TICKS} тиков) ...", end="", flush=True)
for _ in range(WARMUP_TICKS): world.tick()
for q in list(cam_queues.values()) + [lidar_queue]:
    while not q.empty():
        try: q.get_nowait()
        except queue.Empty: break
print(" готово.")

# — CSV —————
csv_file = open(CSV_PATH, 'w', newline='', encoding='utf-8')
writer = csv.writer(csv_file)
writer.writerow(['frame_id', 'image_path', 'lidar_path',
                 'steer', 'throttle', 'brake', 'speed_kmh', 'command', 'cam'])

EMPTY_BEV = np.zeros((HEIGHT_SLICES + 1, BEV_SIZE, BEV_SIZE), dtype=np.float32)

ticks_needed = TOTAL_FRAMES // len(CAM_CONFIGS)
print(f"Сбор {TOTAL_FRAMES} строк = {ticks_needed} тиков езды | {TOWN}")
print(f"Камеры: C / L(+{STEER_CORRECTION:.2f}) / R(-{STEER_CORRECTION:.2f})\n")

cmd_counts = {0: 0, 1: 0, 2: 0, 3: 0}
frame_idx = 0 # глобальный счётчик кадров (строк CSV)
tick_idx = 0 # счётчик тиков с данными лидара (для имени .npy)
saved = 0 # сколько строк сохранено
stuck_ticks = 0
diag_shown = False

try:
    for i in tqdm(range(ticks_needed * 4), desc="Сбор"):
        if saved >= TOTAL_FRAMES: break
        if not vehicle.is_alive:
            print("\nМашина уничтожена."); break

        # — До тика —————
        t_before = vehicle.get_transform()
        loc_before = t_before.location
        yaw_before = t_before.rotation.yaw
        cmd = get_command(vehicle, world_map)

        world.tick()
        weather_tick += 1

        # — После тика —————
        snapshot = world.get_snapshot()
        actor_snap = snapshot.find(vehicle.id)
        t_after = actor_snap.get_transform() if actor_snap else vehicle.get_transform()

        speed_kmh, steer, throttle, brake = compute_controls(
            loc_before, yaw_before, t_after.location, t_after.rotation.yaw)

        # — Детектор застревания → перезапуск автопилота (без телепорта) —
        if speed_kmh < STUCK_KMH:
            stuck_ticks += 1
            if stuck_ticks >= STUCK_TICKS_MAX:
                tqdm.write("⚠ Застыли 15 сек → перезапуск автопилота")
                vehicle.set_autopilot(False, TM_PORT)
                for _ in range(10): world.tick()
                vehicle.set_autopilot(True, TM_PORT)
                tm.vehicle_percentage_speed_difference(vehicle, -50)
                stuck_ticks = 0
            for q in list(cam_queues.values()) + [lidar_queue]:
                while not q.empty():
                    try: q.get_nowait()
                    except queue.Empty: break
            continue
        else:
            stuck_ticks = 0

        if speed_kmh < MIN_SPEED_KMH:
            continue

        # — Смена погоды —————

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

if weather_tick % WEATHER_INTERVAL == 0:
    apply_weather(world)

# — Получить кадры всех трёх камер —————
frames = {}
ok = True
for name in [c[0] for c in CAM_CONFIGS]:
    try:
        frames[name] = cam_queues[name].get(timeout=2.0)
    except queue.Empty:
        tqdm.write(f" WARNING: нет кадра камеры {name} на тике {i}")
        ok = False; break
if not ok: continue

# — LiDAR —————
bev = EMPTY_BEV.copy()
try:
    ld = lidar_queue.get(timeout=1.0)
    pts = np.frombuffer(ld.raw_data, dtype=np.float32).reshape(-1, 4)
    bev = lidar_to_bev(pts)
    while not lidar_queue.empty():
        try:
            ld = lidar_queue.get_nowait()
            bev = lidar_to_bev(
                np.frombuffer(ld.raw_data, dtype=np.float32).reshape(-1, 4))
        except queue.Empty: break
except queue.Empty:
    pass # LiDAR не критичен

# — Сохранить LiDAR (один на тик) —————
lname = f"lidar_{tick_idx:05d}.npy"
np.save(os.path.join(LIDAR_DIR, lname), bev)

# — Сохранить 3 кадра + 3 строки CSV —————
for cam_name, _, steer_corr in CAM_CONFIGS:
    corrected_steer = float(np.clip(steer + steer_corr, -1.0, 1.0))

    raw = np.frombuffer(frames[cam_name].raw_data, dtype=np.uint8)
    img = raw.reshape((IMAGE_SIZE[1], IMAGE_SIZE[0], 4))[:, :, :3]
    img_out = cv2.resize(cv2.cvtColor(img, cv2.COLOR_RGB2BGR), SAVE_SIZE)

    fname = f"frame_{frame_idx:05d}.png"
    cv2.imwrite(os.path.join(IMAGE_DIR, fname), img_out,
                [cv2.IMWRITE_PNG_COMPRESSION, 3])

    writer.writerow([
        frame_idx, fname, lname,
        round(corrected_steer, 4),
        round(float(throttle), 4),
        round(float(brake), 4),
        round(float(speed_kmh), 2),
        cmd, cam_name,
    ])
    cmd_counts[cmd] += 1
    frame_idx += 1
    saved += 1

# — Диагностика первых 2 тиков —————
if tick_idx < 2 and not diag_shown:
    tqdm.write(
        f" [тик {tick_idx}] spd={speed_kmh:.1f} km/h "
        f"steer(C)={steer:.3f} "
        f"steer(L)={steer+STEER_CORRECTION:.3f} "
        f"steer(R)={steer-STEER_CORRECTION:.3f} "
        f"cmd={CMD_NAMES[cmd]} "
        f"lidar={'✓' if bev.max()>0 else 'X'}"
    )
    if tick_idx == 1: diag_shown = True

    tick_idx += 1
    csv_file.flush()

except KeyboardInterrupt:
    print("\n[!] Прервано.")
finally:
    for cam in cam_actors.values():
        cam.stop(); cam.destroy()
    lidar_sensor.stop(); lidar_sensor.destroy()
    vehicle.destroy()

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

        for npc in npcs:
            try: npc.destroy()
            except Exception: pass
        tm.set_synchronous_mode(False)
        csv_file.close()
        s = world.get_settings()
        s.synchronous_mode = False
        world.apply_settings(s)
        print("    Синхронный режим отключён.")

print(f"\n❑ Сохранено: {saved} строк ({tick_idx} тиков)")
print(f"    Кадров реальной езды: {tick_idx} * 3 камеры = {saved}")
print(f"    LiDAR-файлов: {tick_idx}")
print(f"❏ Распределение команд (по всем 3 камерам):")
total = max(1, saved)
for k, v in cmd_counts.items():
    bar = '█' * int(30 * v / total)
    print(f"    {CMD_NAMES[k]:8s}: {v:5d} ({100*v/total:.1f}%) {bar}")

if __name__ == '__main__':
    import carla
    main()

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

ПРИЛОЖЕНИЕ Г

Листинг модуля train.py (обучение модели)

```
"""
Train CILDriveNet on Town10HD data.

CIL training: all 4 command branches are computed each forward pass,
but the loss is applied ONLY to the branch matching the recorded command
(branch-gating from CORL-2017 paper). The camera backbone starts with a
lower learning rate (fine-tuning a pretrained ResNet-18).

Usage:
python scripts/train.py
python scripts/train.py --data data/train --epochs 60 --batch 16
"""

import os
import sys
import time
import argparse
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.tensorboard import SummaryWriter

sys.path.insert(0, os.path.dirname(__file__))
from model import CILDriveNet
from dataset import get_data loaders

# — Loss —————

def cil_loss(
    all_preds: torch.Tensor, # (B, 4, 2)
    targets: torch.Tensor, # (B, 2) [steer, throttle]
    commands: torch.Tensor, # (B,) int64
    steer_w: float = 8.0,
):
    """
    CIL branch-gated loss с per-sample весами:
    - steer_w=8 (был 5): руль критичнее газа
    - поворотные ветки (LEFT/RIGHT/STRAIGHT) получают ×2 по стире —
      они обучались на малом числе семплов и без доп. давления недообучаются
    """
    B = all_preds.size(0)
    device = all_preds.device
    pred = all_preds[torch.arange(B, device=device), commands] # (B, 2)

    steer_loss_ps = nn.functional.smooth_l1_loss(pred[:, 0], targets[:, 0],
                                                  reduction='none') # (B,)
    throt_loss_ps = nn.functional.mse_loss(pred[:, 1], targets[:, 1],
                                           reduction='none') # (B,)

    # Для поворотных веток (cmd ≠ FOLLOW) удваиваем вес потери по стире.
    # FOLLOW-ветка и без того доминирует по числу семплов.
    turn_mask = (commands != 0).float() # 0 или 1
    steer_w_ps = steer_w * (1.0 + turn_mask) # 8 или 16

    per_sample = steer_w_ps * steer_loss_ps + throt_loss_ps
    total = per_sample.mean()

    return total, steer_loss_ps.mean().item(), throt_loss_ps.mean().item()

# — Train / validate loops —————

SPEED_LOSS_W = 0.5 # вес auxiliary speed-loss; 0 = отключить

def train_epoch(model, loader, optimizer, scaler, device):
    model.train()
    tot = s = t = sp = 0.0
    for imgs, lidars, speeds, commands, targets in loader:
        imgs = imgs.to(device, non_blocking=True)
```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

        lidars = lidars.to(device, non_blocking=True)
        speeds = speeds.to(device, non_blocking=True)
        commands = commands.to(device, non_blocking=True)
        targets = targets.to(device, non_blocking=True)

        optimizer.zero_grad()
        with torch.cuda.amp.autocast(enabled=device.type == 'cuda'):
            # h=None: при обучении скрытое состояние сбрасывается на каждый батч
            preds, speed_pred, _ = model(imgs, lidars, speeds, h=None)
            loss, sl, tl = cil_loss(preds, targets, commands)
            spd_loss = nn.functional.mse_loss(speed_pred, speeds)
            loss = loss + SPEED_LOSS_W * spd_loss

        scaler.scale(loss).backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        scaler.step(optimizer)
        scaler.update()

        tot += loss.item()
        s += sl
        t += tl
        sp += spd_loss.item()

    n = len(loader)
    return tot / n, s / n, t / n, sp / n

@torch.no_grad()
def validate(model, loader, device):
    model.eval()
    tot = s = t = sp = 0.0
    cmd_losses = {0: [], 1: [], 2: [], 3: []}

    for imgs, lidars, speeds, commands, targets in loader:
        imgs = imgs.to(device, non_blocking=True)
        lidars = lidars.to(device, non_blocking=True)
        speeds = speeds.to(device, non_blocking=True)
        targets = targets.to(device, non_blocking=True)
        cmds_np = commands.numpy().copy()
        commands = commands.to(device, non_blocking=True)

        preds, speed_pred, _ = model(imgs, lidars, speeds, h=None)
        loss, sl, tl = cil_loss(preds, targets, commands)
        spd_loss = nn.functional.mse_loss(speed_pred, speeds)
        tot += loss.item(); s += sl; t += tl; sp += spd_loss.item()

        B = preds.size(0)
        pred = preds[torch.arange(B, device=device), commands]
        for i in range(B):
            l = nn.functional.smooth_l1_loss(pred[i], targets[i]).item()
            cmd_losses[int(cmds_np[i])].append(l)

    n = len(loader)
    cmean = {k: float(np.mean(v)) if v else 0.0 for k, v in cmd_losses.items()}
    return tot / n, s / n, t / n, sp / n, cmean

# — Main —

def main(args):
    torch.manual_seed(42)
    np.random.seed(42)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(42)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"🚗 Device: {device}")

    trn_loader, val_loader = get_dataloaders(
        args.data, batch_size=args.batch, num_workers=args.workers
    )

    model = CILDriveNet().to(device)

    # Differential learning rates: backbone 10x lower than head
    backbone_params = list(model.camera_encoder.parameters())
    other_params = (
        list(model.lidar_encoder.parameters()) +
        list(model.speed_encoder.parameters()) +

```

отформатировано: английский (США)

отформатировано: английский (США)

```

        list(model.fusion.parameters()) +
        list(model.gru.parameters()) +
        list(model.gru_dropout.parameters()) +
        list(model.speed_head.parameters()) +
        [p for branch in model.branches for p in branch.parameters()]
    )
    optimizer = optim.AdamW([
        {'params': backbone_params, 'lr': args.lr * 0.1},
        {'params': other_params, 'lr': args.lr},
    ], weight_decay=1e-4)

    scheduler = optim.lr_scheduler.CosineAnnealingWarmRestarts(
        optimizer, T_0=20, T_mult=2, eta_min=1e-6
    )
    scaler = torch.cuda.amp.GradScaler(enabled=device.type == 'cuda')
    writer = SummaryWriter(log_dir=os.path.join(args.logdir, "tensorboard"))

    best_val = float("inf")
    no_improv = 0
    os.makedirs(args.ckpt_dir, exist_ok=True)

    print("🚀 Training CILDriveNet ...")
    for epoch in range(1, args.epochs + 1):
        t0 = time.time()

        tr_loss, tr_s, tr_t, tr_sp = train_epoch(model, trn_loader, optimizer, scaler,
device)
        vl_loss, vl_s, vl_t, vl_sp, cmd = validate(model, val_loader, device)
        scheduler.step(epoch)

        # TensorBoard
        writer.add_scalars("Loss", {"train": tr_loss, "val": vl_loss}, epoch)
        writer.add_scalars("Steer", {"train": tr_s, "val": vl_s}, epoch)
        writer.add_scalars("Throttle", {"train": tr_t, "val": vl_t}, epoch)
        writer.add_scalars("SpeedPred", {"train": tr_sp, "val": vl_sp}, epoch)
        for c, l in cmd.items():
            writer.add_scalar(f"Val/cmd_{c}", l, epoch)
        writer.flush()

        elapsed = time.time() - t0
        print(
            f"Epoch {epoch:03d} | "
            f"Train {tr_loss:.4f} Val {vl_loss:.4f} | "
            f"S {vl_s:.4f} T {vl_t:.4f} Spd {vl_sp:.4f} | "
            f"L {cmd[1]:.4f} R {cmd[2]:.4f} Str {cmd[3]:.4f} | "
            f"{elapsed:.1f}s"
        )

        if vl_loss < best_val:
            best_val = vl_loss
            no_improv = 0
            ckpt_path = os.path.join(args.ckpt_dir, "best_model.pth")
            torch.save({
                "epoch": epoch,
                "model_state_dict": model.state_dict(),
                "optimizer_state_dict": optimizer.state_dict(),
                "val_loss": vl_loss,
                "cmd_losses": cmd,
            }, ckpt_path)
            print(f"🏆 New best → {ckpt_path}")
        else:
            no_improv += 1
            if no_improv >= args.patience:
                print(f"🛑 Early stopping at epoch {epoch}.")
                break

    writer.close()
    print(f"\n🏁 Training complete. Best val_loss: {best_val:.4f}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--data", default="data/train", help="Data directory")
    parser.add_argument("--epochs", type=int, default=80)
    parser.add_argument("--batch", type=int, default=32)
    parser.add_argument("--lr", type=float, default=3e-4)
    parser.add_argument("--patience", type=int, default=20)
    parser.add_argument("--workers", type=int, default=2)
    parser.add_argument("--ckpt_dir", default="checkpoints")

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```
parser.add_argument("--logdir", default="logs")  
main(parser.parse_args())
```


ПРИЛОЖЕНИЕ Д

Листинг модуля dataset.py (загрузка данных и формирование последовательностей)

```
"""
CarlaDataset — загружает последовательности кадров для GRU-обучения.

Каждый сэмпл содержит:
    imgs_seq (seq_len, 3, 224, 224) — Т последовательных кадров одной камеры
    lidar_bev (5, 200, 200) — LiDAR текущего (последнего) кадра
    speed_t (1,) — скорость текущего кадра
    command int — навигационная команда текущего кадра
    targets (2,) — [steer, throttle] текущего кадра

Последовательность строится из строк CSV с тем же 'cam' (C/L/R).
Если предыдущих кадров не хватает — повторяем самый ранний доступный.

seq_len=1 → поведение как у старого датасета (без GRU).
"""

import os
import numpy as np
import pandas as pd
import cv2
import torch
from torch.utils.data import Dataset, DataLoader, WeightedRandomSampler

from utils import IMG_MEAN, IMG_STD, MAX_SPEED_KMH

SEQ_LEN = 4 # кадров в последовательности (0.2 сек при 20 FPS)

class CarlaDataset(Dataset):
    def __init__(self, csv_path: str, data_dir: str,
                 train: bool = True, seq_len: int = SEQ_LEN):
        self.df = pd.read_csv(csv_path)
        self.img_dir = os.path.join(data_dir, "images")
        self.lidar_dir = os.path.join(data_dir, "lidar")
        self.train = train
        self.seq_len = seq_len

        # Проверяем наличие колонки cam (обратная совместимость со старым датасетом)
        if 'cam' not in self.df.columns:
            self.df['cam'] = 'C'

        # Индексы строк для каждой камеры: {'C': [0,3,6,...], 'L': [1,4,...], 'R': [2,5,...]}
        self._cam_idx = {}
        for cam in self.df['cam'].unique():
            self._cam_idx[cam] = list(self.df.index[self.df['cam'] == cam])

        # Позиция каждой строки внутри своей камеры: row_i → pos внутри cam-группы
        self._row_pos = {}
        for cam, indices in self._cam_idx.items():
            for pos, idx in enumerate(indices):
                self._row_pos[idx] = pos

        # — Sampling weights —
        w = torch.ones(len(self.df), dtype=torch.float32)
        w[self.df['steer'].abs() > 0.06] *= 8.0
        w[self.df['command'].isin([1, 2, 3])] *= 6.0
        self.weights = w

    def __len__(self) -> int:
        return len(self.df)

    def _load_image(self, row) -> np.ndarray:
        img_bgr = cv2.imread(os.path.join(self.img_dir, row['image_path']))
        return cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB).astype(np.float32)

    def _augment(self, img: np.ndarray, lidar_bev: np.ndarray,
                 steer: float, command: int):
        """Аугментации применяются одинаково ко всем кадрам последовательности."""
        # Горизонтальный флип (одно решение на всю последовательность)
        do_flip = self.train and np.random.random() > 0.5
```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

# Цветовое дрожание (отдельное для каждого кадра)
alpha = np.random.uniform(0.80, 1.20) if self.train else 1.0
beta = np.random.uniform(-20.0, 20.0) if self.train else 0.0
do_hsv = self.train and np.random.random() > 0.6
do_lidar_drop = self.train and np.random.random() > 0.8
return do_flip, alpha, beta, do_hsv, do_lidar_drop

def _process_image(self, img: np.ndarray, do_flip: bool,
                  alpha: float, beta: float, do_hsv: bool) -> torch.Tensor:
    if do_flip:
        img = img[:, ::-1, :].copy()
    img = np.clip(img * alpha + beta, 0.0, 255.0)
    if do_hsv:
        hsv = cv2.cvtColor(img.astype(np.uint8), cv2.COLOR_RGB2HSV).astype(np.float32)
        hsv[:, :, 2] *= np.random.uniform(0.70, 1.30)
        hsv[:, :, 2] = np.clip(hsv[:, :, 2], 0, 255)
        img = cv2.cvtColor(hsv.astype(np.uint8), cv2.COLOR_HSV2RGB).astype(np.float32)
    img /= 255.0
    img = (img - IMG_MEAN) / IMG_STD
    return torch.from_numpy(img.astype(np.float32)).permute(2, 0, 1) # (3,H,W)

def __getitem__(self, idx: int):
    row = self.df.iloc[idx]
    cam = row['cam']
    cam_idx = self._cam_idx[cam] # все индексы этой камеры
    pos = self._row_pos[idx] # позиция текущей строки в камере

    # — Построить последовательность индексов —
    # [pos-(T-1), ..., pos-1, pos], клэмпим на 0
    seq_positions = [max(0, pos - (self.seq_len - 1 - k))
                    for k in range(self.seq_len)]
    seq_df_indices = [cam_idx[p] for p in seq_positions]

    # — Определить аугментации (одни для всей последовательности) —
    cur_row = self.df.iloc[idx]
    steer = float(cur_row['steer'])
    command = int(cur_row['command'])
    do_flip, alpha, beta, do_hsv, do_lidar_drop = self._augment(
        None, None, steer, command)

    # — Загрузить и обработать изображения последовательности —
    imgs = []
    for si in seq_df_indices:
        img_raw = self._load_image(self.df.iloc[si])
        img_t = self._process_image(img_raw, do_flip, alpha, beta, do_hsv)
        imgs.append(img_t)
    imgs_seq = torch.stack(imgs, dim=0) # (seq_len, 3, H, W)

    # — LiDAR текущего кадра —
    lidar_bev = np.load(
        os.path.join(self.lidar_dir, cur_row['lidar_path'])
    ).astype(np.float32)
    if do_flip:
        lidar_bev = lidar_bev[:, :, ::-1].copy()
    if do_lidar_drop:
        lidar_bev[np.random.random(lidar_bev.shape) < 0.05] = 0.0
    lidar_t = torch.from_numpy(lidar_bev) # (5, 200, 200)

    # — Метки текущего кадра —
    speed = float(cur_row['speed_kmh']) / MAX_SPEED_KMH
    if do_flip:
        steer = -steer
        if command == 1: command = 2
        elif command == 2: command = 1

    speed_t = torch.tensor([np.clip(speed, 0.0, 1.0)], dtype=torch.float32)
    targets = torch.tensor([steer, float(cur_row['throttle'])],
                          dtype=torch.float32)

    return imgs_seq, lidar_t, speed_t, command, targets

def get_dataloaders(data_dir: str, batch_size: int = 16,
                   val_split: float = 0.15, num_workers: int = 2,
                   seq_len: int = SEQ_LEN):
    csv_path = os.path.join(data_dir, "labels.csv")

    trn_full = CarlaDataset(csv_path, data_dir, train=True, seq_len=seq_len)
    val_full = CarlaDataset(csv_path, data_dir, train=False, seq_len=seq_len)

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

n            = len(trn_full)
trn_size = int(n * (1 - val_split))

trn_idx = list(range(trn_size))
val_idx = list(range(trn_size, n))

trn_ds = torch.utils.data.Subset(trn_full, trn_idx)
val_ds = torch.utils.data.Subset(val_full, val_idx)

trn_weights = trn_full.weights[torch.tensor(trn_idx)]
sampler     = WeightedRandomSampler(trn_weights, len(trn_ds), replacement=True)

trn_loader = DataLoader(trn_ds, batch_size=batch_size, sampler=sampler,
                        num_workers=num_workers, pin_memory=True, drop_last=True)
val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False,
                        num_workers=num_workers, pin_memory=True)

df = trn_full.df
print(f"Dataset: {n} строк, ({trn_size} train / {n-trn_size} val) | seq_len={seq_len}")
for cmd_id, name in {0:"FOLLOW",1:"LEFT",2:"RIGHT",3:"STRAIGHT"}.items():
    cnt = (df['command'] == cmd_id).sum()
    print(f"    {name:8s}: {cnt:5d} ({100*cnt/n:.1f}%)")

return trn_loader, val_loader

```

отформатировано: английский (США)

ПРИЛОЖЕНИЕ Е

Листинг модуля inference.py (инференс в реальном времени)

```
"""
CILDriveNet inference on CARLA Town01.

The model receives:
- resized+normalised camera frame
- LIDAR bird's-eye-view
- normalised vehicle speed
- navigation command inferred from the road geometry ahead

Control outputs are smoothed and a two-layer safety brake is applied:
1. sensor.other.obstacle - CARLA collision sensor (vehicles & pedestrians)
2. obstacle_ahead(pts) - LIDAR point-cloud fallback for static objects

Usage:
python scripts/inference.py
python scripts/inference.py --ckpt checkpoints/best_model.pth --town Town01
"""

import os
import sys
import time
import queue
import random
import argparse
import numpy as np
import cv2
import torch

sys.path.insert(0, os.path.dirname(__file__))
from model import CILDriveNet
from utils import (
    lidar_to_bev, get_command, obstacle_ahead,
    IMG_MEAN, IMG_STD, IMG_SIZE, MAX_SPEED_KMH,
    CMD_NAMES, CMD_FOLLOW, CMD_LEFT, CMD_RIGHT, CMD_STRAIGHT,
    BEV_SIZE, HEIGHT_SLICES,
)

# --- Config ---
FPS = 20
DELTA_SEC = 1.0 / FPS
TARGET_SPD = 30.0 # km/h cruise speed
STEER_SM = 0.35 # сглаживание руля (меньше -> быстрее реакция)
THROT_SM = 0.65 # throttle smoothing
RECORD = True
OUT_VIDEO = "demo_cil.mp4"

# Pre-junction deceleration - тормозим за 28 м, въезжаем на ~14 км/ч.
# get_command() теперь активирует LEFT/RIGHT уже с 22 м, так что
# модель успевает начать поворот при низкой скорости.
JUNCTION_DECEL_M = 28.0
JUNCTION_SPD = 14.0

# --- Junction lookahead ---

def junction_ahead(vehicle, world_map, lookahead_m: float = 28.0) -> bool:
    """
    True если впереди в радиусе lookahead_m метров есть перекрёсток.
    Проверяем несколько точек вдоль дороги, потому что wp.next(d) при большом d
    может «проскочить» через короткие junction-зоны.
    """
    import carla
    loc = vehicle.get_location()
    wp = world_map.get_waypoint(loc, project_to_road=True,
                                lane_type=carla.LaneType.Driving)

    if wp is None:
        return False
    if wp.is_junction:
        return True
    step = lookahead_m / 4.0 # 4 промежуточные точки
    cur = wp
    for _ in range(4):
```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

        nxt = cur.next(step)
        if not nxt:
            break
        cur = nxt[0]
        if cur.is_junction:
            return True
        return False

# — Preprocessing —————
def preprocess_image(img_bgr: np.ndarray) -> torch.Tensor:
    img = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB).astype(np.float32)
    img = cv2.resize(img, IMG_SIZE)
    img = img / 255.0
    img = (img - IMG_MEAN) / IMG_STD
    return torch.from_numpy(img).permute(2, 0, 1).unsqueeze(0).float()

# — HUD overlay —————
def draw_hud(frame, steer, throttle, speed, cmd, braking):
    h, w = frame.shape[:2]
    cv2.putText(frame, f"CMD: {CMD_NAMES[cmd]}", (10, 32),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)
    cv2.putText(frame, f"S: {steer:.3f} T: {throttle:.3f}",
                (10, 64), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
    cv2.putText(frame, f"V: {speed:.1f} km/h", (10, 96),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 200, 0), 2)
    if braking:
        cv2.putText(frame, "!! OBSTACLE !!", (10, 130),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255), 3)
    cv2.putText(frame, "CILDriveNet v2 (Town01)",
                (10, h - 14), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (200, 200, 200), 2)

# — Main —————
def main(args):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"🚗 Device: {device}")

    # Load model
    model = CILDriveNet().to(device)
    ckpt = torch.load(args.ckpt, map_location=device)
    model.load_state_dict(ckpt['model_state_dict'])
    model.eval()
    print(f"📦 Model loaded (epoch {ckpt['epoch']}, val_loss {ckpt['val_loss']:.4f})")
    if 'cmd_losses' in ckpt:
        cl = ckpt['cmd_losses']
        print(f"    Branch losses — L:{cl[1]:.4f} R:{cl[2]:.4f} Str:{cl[3]:.4f}")

    import carla

    print("Connecting to CARLA ...")
    client = carla.Client('127.0.0.1', 2000)
    client.set_timeout(30.0)
    world = client.get_world()

    current = world.get_map().name.split('/')[1]
    print(f"    Current map: {current}")
    if args.town and current != args.town:
        print(f"    Switching {current} -> {args.town} ...")
        world = client.load_world(args.town)
        import time; time.sleep(3.0)
        world = client.get_world()
        print(f"    Map loaded: {world.get_map().name}")

    world_map = world.get_map()
    settings = world.get_settings()
    settings.synchronous_mode = True
    settings.fixed_delta_seconds = DELTA_SEC
    world.apply_settings(settings)

    bp_lib = world.get_blueprint_library()
    spawn_points = world_map.get_spawn_points()

    # Ego vehicle
    v_bp = bp_lib.find('vehicle.tesla.model3')
```

отформатировано: английский (США)

```

vehicle = None
for sp in random.sample(spawn_points, min(15, len(spawn_points))):
    vehicle = world.try_spawn_actor(v_bp, sp)
    if vehicle:
        break
if not vehicle:
    print("✗ Could not spawn vehicle.")
    return
vehicle.set_autopilot(False)

# Camera - параметры должны совпадать с collect_data.py
cam_bp = bp_lib.find('sensor.camera.rgb')
cam_bp.set_attribute('image_size_x', '640')
cam_bp.set_attribute('image_size_y', '480')
cam_bp.set_attribute('fov', '90')
cam_bp.set_attribute('enable_postprocess_effects', 'False')
cam = world.spawn_actor(
    cam_bp,
    carla.Transform(carla.Location(x=2.5, z=1.0), carla.Rotation(yaw=0)),
    attach_to=vehicle
)
cam_q = queue.Queue(maxsize=2)
def _cam_cb(img):
    if cam_q.full():
        try: cam_q.get_nowait()
        except queue.Empty: pass
    cam_q.put_nowait(img)
cam.listen(_cam_cb)

# LIDAR - идентичные параметры collect_data.py (иначе BEV будет другим)
lidar_bp = bp_lib.find('sensor.lidar.ray_cast')
lidar_bp.set_attribute('channels', '32')
lidar_bp.set_attribute('range', '40')
lidar_bp.set_attribute('points_per_second', '56000')
lidar_bp.set_attribute('rotation_frequency', str(FPS))
lidar_bp.set_attribute('upper_fov', '10.0')
lidar_bp.set_attribute('lower_fov', '-30.0')
lidar = world.spawn_actor(
    lidar_bp,
    carla.Transform(carla.Location(x=0.0, z=2.4)),
    attach_to=vehicle
)
lidar_q = queue.Queue(maxsize=2)
def _lidar_cb(data):
    if lidar_q.full():
        try: lidar_q.get_nowait()
        except queue.Empty: pass
    lidar_q.put_nowait(data)
lidar.listen(_lidar_cb)

# — Obstacle sensor (sensor.other.obstacle) —————
# Приоритетный датчик препятствий: работает по физическому лучу, не по LIDAR.
# Дополняет LIDAR-based obstacle ahead() для статических объектов.
obs_bp = bp_lib.find('sensor.other.obstacle')
obs_bp.set_attribute('distance', '15') # дальность обнаружения, м
obs_bp.set_attribute('hit_radius', '0.5') # радиус луча
obs_bp.set_attribute('only_dynamics', 'False') # ловит и статику, и динамику
obs_sensor = world.spawn_actor(
    obs_bp,
    carla.Transform(), # без смещения — на позиции машины
    attach_to=vehicle
)

# Простой thread-safe флаг; сбрасывается каждые N кадров
obstacle_flag = {'detected': False}

def _obs_cb(event):
    atype = event.other_actor.type_id
    if 'vehicle' in atype or 'walker' in atype:
        obstacle_flag['detected'] = True

obs_sensor.listen(_obs_cb)

# Video writer
vw = None
if RECORD:
    vw = cv2.VideoWriter(OUT_VIDEO, cv2.VideoWriter_fourcc(*'mp4v'),
                        FPS, (640, 480))

# Warm-up

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

for _ in range(30):
    world.tick()

prev_steer = 0.0
prev_throt = 0.0
frame_cnt = 0
OBSTACLE_GRACE = 60 # первые 3 секунды (60 кадров) пропускаем obstacle check

print("Inference running. Press 'q' to quit.")

try:
    while True:
        world.tick()

        try:
            cam_data = cam_q.get(timeout=0.5)
            lidar_data = lidar_q.get(timeout=0.5)
        except queue.Empty:
            continue

        # --- Sensor data ---
        raw = np.frombuffer(cam_data.raw_data, dtype=np.uint8)
        img_bgr = cv2.cvtColor(
            raw.reshape(480, 640, 4)[: , : , :3], cv2.COLOR_RGB2BGR
        )
        pts = np.frombuffer(lidar_data.raw_data, dtype=np.float32).reshape(-1, 4)
        bev = lidar_to_bev(pts)

        # --- Vehicle state ---
        vel = vehicle.get_velocity()
        speed = 3.6 * (vel.x**2 + vel.y**2 + vel.z**2) ** 0.5
        spd_norm = float(np.clip(speed / MAX_SPEED_KMH, 0.0, 1.0))
        cmd = get_command(vehicle, world_map)

        # --- Model inference ---
        with torch.no_grad():
            img_t = preprocess_image(img_bgr).to(device)
            lidar_t = torch.from_numpy(bev).unsqueeze(0).to(device)
            spd_t = torch.tensor([spd_norm], dtype=torch.float32).to(device)

            all_branches, _ = model(img_t, lidar_t, spd_t) # (1,4,2); speed_pred ignored
            steer_raw, throt_raw = all_branches[0, cmd].cpu().numpy()

        steer_raw = float(steer_raw)
        throt_raw = float(throt_raw)

        # --- Smooth control ---
        steer = STEER_SM * prev_steer + (1 - STEER_SM) * steer_raw
        throt = THROT_SM * prev_throt + (1 - THROT_SM) * throt_raw

        # Speed governor (крейсерская скорость)
        if speed > TARGET_SPD:
            excess = (speed - TARGET_SPD) / TARGET_SPD
            throt = max(0.0, throt * (1.0 - excess))

        steer = float(np.clip(steer, -1.0, 1.0))
        throt = float(np.clip(throt, 0.0, 1.0))
        brake = 0.0

        # --- Торможение перед перекрёстком ---
        junction_near = _junction_ahead(vehicle, world_map, JUNCTION_DECEL_M)
        if junction_near:
            throt = min(throt, 0.30)
            if speed > JUNCTION_SPD:
                brake = 0.25

        # --- Safety brake: obstacle sensor + LIDAR fallback ---
        obs_sensor_hit = obstacle_flag['detected']
        obstacle_flag['detected'] = False
        lidar_hit = obstacle_ahead(pts)

        # grace period: первые OBSTACLE_GRACE кадров не тормозим по датчикам -
        # на спавне LIDAR часто видит столбы/знаки и даёт ложный brake=0.8
        emerg = (obs_sensor_hit or lidar_hit) and (frame_cnt >= OBSTACLE_GRACE)
        if emerg:
            throt = 0.0
            brake = 0.8

        # при скорости < 2 км/ч модель сама не трогается (throttle bias низкий)

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

if speed < 2.0 and not emerg:
    throt = max(throt, 0.15)

if frame_cnt < 5:
    print(f"[DBG #{frame_cnt}] cmd={CMD_NAMES[cmd]} "
          f"steer_raw={steer_raw:.3f} throt_raw={throt_raw:.3f} "
          f"throt={throt:.3f} speed={speed:.1f} "
          f"obs_sensor={obs_sensor_hit} lidar={lidar_hit} emerg={emerg}")

prev_steer = steer
prev_throt = throt

# Apply
vehicle.apply_control(carla.VehicleControl(
    throttle=throt, steer=steer, brake=brake,
    manual_gear_shift=False,
))

# — Display —————
draw_hud(img_bgr, steer, throt, speed, cmd, emerg)
if vw:
    vw.write(img_bgr)
cv2.imshow("CIL Navigation", img_bgr)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

frame_cnt += 1
if frame_cnt % 100 == 0:
    print(f"[{frame_cnt:5d}] CMD={CMD_NAMES[cmd]:8s} "
          f"S={steer:.3f} T={throt:.3f} V={speed:.1f} km/h")

except KeyboardInterrupt:
    pass
finally:
    cam.stop();          cam.destroy()
    lidar.stop();        lidar.destroy()
    obs_sensor.stop();   obs_sensor.destroy()
    vehicle.destroy()
    if vw:
        vw.release()
        print(f"📹 Video saved to {OUT_VIDEO}")
    cv2.destroyAllWindows()
    s = world.get_settings()
    s.synchronous_mode = False
    world.apply_settings(s)
    print("🏁 Done.")

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--ckpt", default="checkpoints/best_model.pth")
    parser.add_argument("--town", default="Town10HD")
    main(parser.parse_args())

```

отформатировано: английский (США)

отформатировано: английский (США)

ПРИЛОЖЕНИЕ Ж

Листинг модуля inference_test.py (тестирование с трафиком)

```
"""
CILDriveNet — тестовый инференс с трафиком.

Отличия от inference.py:
- Спавнится N NPC-машин под управлением ТМ
- Датчик столкновения: при аварии машина телепортируется на новую точку и продолжает
- Счётчик столкновений выводится в HUD
- Автоматический respawn — тест продолжается даже после аварии

Управление:
Q — выход
R — ручной respawn (телепорт в новую точку)

Запуск:
python scripts/inference_test.py
python scripts/inference_test.py --ckpt checkpoints/best_model.pth --town Town03
"""

import os
import sys
import time
import queue
import random
import argparse
import numpy as np
import cv2
import torch

sys.path.insert(0, os.path.dirname(__file__))
from model import CILDriveNet
from utils import (
    lidar_to_bev, get_command, obstacle_ahead,
    IMG_MEAN, IMG_STD, IMG_SIZE, MAX_SPEED_KMH,
    CMD_NAMES,
)

# — Конфиг —————

FPS = 20
DELTA_SEC = 1.0 / FPS
TM_PORT = 8000

TARGET_SPD = 30.0 # км/ч — крейсерская скорость
STEER_SM = 0.35 # сглаживание руля
THROT_SM = 0.60 # сглаживание газа
NPC_COUNT = 30 # машин, трафика

JUNCTION_DECEL_M = 28.0
JUNCTION_SPD = 12.0 # км/ч — скорость въезда на перекрёсток

RECORD = True
OUT_VIDEO = "demo_cil_test.mp4"

RESPAWN_AFTER_COLLISION = True # автоматически возобновлять после аварии
RESPAWN_PAUSE_TICKS = 60 # тиков паузы перед respawn (3 сек)

# — Preprocessing —————

def preprocess_image(img_bgr: np.ndarray) -> torch.Tensor:
    img = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB).astype(np.float32) / 255.0
    img = cv2.resize(img, IMG_SIZE)
    img = (img - IMG_MEAN) / IMG_STD
    return torch.from_numpy(img).permute(2, 0, 1).unsqueeze(0).float()

# — Junction lookahead —————

def junction_ahead(vehicle, world_map, lookahead_m=28.0) -> bool:
    import carla
    loc = vehicle.get_location()
    wp = world_map.get_waypoint(loc, project_to_road=True,
```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

        lane_type=carla.LaneType.Driving)

    if wp is None:
        return False
    if wp.is_junction:
        return True
    step = lookahead_m / 4.0
    cur = wp
    for _ in range(4):
        nxt = cur.next(step)
        if not nxt:
            break
        cur = nxt[0]
        if cur.is_junction:
            return True
    return False

# --- HUD -----
def draw_hud(frame, steer, throttle, speed, cmd, braking, collisions, respawns):
    h, _ = frame.shape[:2]

    # Полупрозрачный фон под текстом
    overlay = frame.copy()
    cv2.rectangle(overlay, (0, 0), (300, 145), (0, 0, 0), -1)
    cv2.addWeighted(overlay, 0.45, frame, 0.55, 0, frame)

    cv2.putText(frame, f"CMD: {CMD_NAMES[cmd]}",
                (10, 28), cv2.FONT_HERSHEY_SIMPLEX, 0.75, (0, 255, 255), 2)
    cv2.putText(frame, f"STR: {steer:+.3f}   THR: {throttle:.3f}",
                (10, 58), cv2.FONT_HERSHEY_SIMPLEX, 0.70, (0, 255, 0), 2)
    cv2.putText(frame, f"SPD: {speed:.1f} km/h",
                (10, 88), cv2.FONT_HERSHEY_SIMPLEX, 0.70, (255, 200, 0), 2)
    cv2.putText(frame, f"COLL: {collisions}   RESP: {respawns}",
                (10, 118), cv2.FONT_HERSHEY_SIMPLEX, 0.65, (180, 180, 180), 2)

    if braking:
        cv2.putText(frame, "!! OBSTACLE !!",
                    (10, h - 40), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255), 3)

    cv2.putText(frame, "CILDiveNet [Q=quit R=respawn]",
                (10, h - 12), cv2.FONT_HERSHEY_SIMPLEX, 0.55, (180, 180, 180), 1)

# --- Respawn эго-машин -----
def respawn_vehicle(vehicle, world, spawn_points, camera, lidar_sensor,
                    img_queue, lidar_queue):
    """Телепортировать машину на свободную точку, сбросить очереди."""
    import carla

    # Ищем spawn-точку, рядом с которой нет других машин
    actors = world.get_actors().filter('vehicle.*')
    actor_locs = [a.get_location() for a in actors if a.id != vehicle.id]

    for sp in random.sample(spawn_points, min(30, len(spawn_points))):
        # Проверяем что в радиусе 8 м нет других машин
        clear = all(
            sp.location.distance(loc) > 8.0
            for loc in actor_locs
        )
        if clear:
            vehicle.set_transform(sp)
            break

    # Сбрасываем очереди сенсоров
    for q in (img_queue, lidar_queue):
        while not q.empty():
            try: q.get_nowait()
            except queue.Empty: break

    # Даём физике устояться
    for _ in range(30):
        world.tick()

# --- Спавн NPC -----
def spawn_npcs(world, tm, count, spawn_points, bp_lib):
    npc_bps = [bp for bp in bp_lib.filter('vehicle.*')]

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

        if int(bp.get_attribute('number_of_wheels')) >= 4]
npcs, used = [], set()
for _ in range(count * 6):
    if len(npcs) >= count:
        break
    sp = random.choice(spawn_points)
    key = (round(sp.location.x, 1), round(sp.location.y, 1))
    if key in used:
        continue
    used.add(key)
    bp = random.choice(npc_bps)
    if bp.has_attribute('color'):
        bp.set_attribute('color', random.choice(
            bp.get_attribute('color').recommended_values))
    npc = world.try_spawn_actor(bp, sp)
    if npc:
        npc.set_autopilot(True, TM_PORT)
        npcs.append(npc)
return npcs

# — Main —
def main(args):
    import carla

    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f"🔥 Device: {device}")

    # — Загрузка модели —
    model = CILDriveNet().to(device)
    ckpt = torch.load(args.ckpt, map_location=device)
    model.load_state_dict(ckpt['model_state_dict'])
    model.eval()
    print(f"🔥 Модель загружена (epoch {ckpt['epoch']}, val loss {ckpt['val loss']:.4f})")

    # — CARLA —
    print(f"🔥 Подключение к CARLA ...")
    client = carla.Client('127.0.0.1', 2000)
    client.set_timeout(30.0)
    world = client.get_world()

    current = world.get_map().name.split('/')[ -1]
    if current != args.town:
        print(f"🔥 {current} → {args.town} ...")
        world = client.load_world(args.town)
        time.sleep(3.0)
        world = client.get_world()
    print(f"🔥 Карта: {world.get_map().name}")

    world_map = world.get_map()
    settings = world.get_settings()
    settings.synchronous_mode = True
    settings.fixed_delta_seconds = DELTA_SEC
    world.apply_settings(settings)

    # — TrafficManager —
    tm = client.get_trafficmanager(TM_PORT)
    tm.set_synchronous_mode(True)
    tm.set_hybrid_physics_mode(False)
    tm.set_global_distance_to_leading_vehicle(2.5)

    bp_lib = world.get_blueprint_library()
    spawn_points = world_map.get_spawn_points()

    # — Спавн NPC —
    print(f"🔥 Спавн {NPC_COUNT} NPC ...")
    npcs = spawn_npcs(world, tm, NPC_COUNT, spawn_points, bp_lib)
    print(f"🔥 NPC: {len(npcs)}")

    # — Эго-машина —
    v_bp = bp_lib.find('vehicle.tesla.model3')
    vehicle = None
    for sp in random.sample(spawn_points, min(20, len(spawn_points))):
        vehicle = world.try_spawn_actor(v_bp, sp)
        if vehicle:
            break
    if not vehicle:

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

        print("❌ Не удалось заspawnить эго-машину.")
        return

# Эго-машина управляется моделью, НЕ автопилотом
vehicle.set_autopilot(False)
print(f"✅ Эго-машина_id={vehicle.id}")

# — Камера —
cam_bp = bp_lib.find('sensor.camera.rgb')
cam_bp.set_attribute('image_size_x', '640')
cam_bp.set_attribute('image_size_y', '480')
cam_bp.set_attribute('fov', '90')
cam_bp.set_attribute('enable_postprocess_effects', 'False')
camera = world.spawn_actor(
    cam_bp,
    carla.Transform(carla.Location(x=2.5, z=1.0), carla.Rotation(yaw=0)),
    attach_to=vehicle)
img_queue = queue.Queue(maxsize=2)
def _cam_cb(img):
    if img_queue.full():
        try: img_queue.get_nowait()
        except queue.Empty: pass
    img_queue.put_nowait(img)
camera.listen(_cam_cb)

# — LIDAR —
lidar_bp = bp_lib.find('sensor.lidar.ray_cast')
lidar_bp.set_attribute('channels', '32')
lidar_bp.set_attribute('range', '40')
lidar_bp.set_attribute('points_per_second', '56000')
lidar_bp.set_attribute('rotation_frequency', str(FPS))
lidar_bp.set_attribute('upper_fov', '10.0')
lidar_bp.set_attribute('lower_fov', '-30.0')
lidar_sensor = world.spawn_actor(
    lidar_bp,
    carla.Transform(carla.Location(x=0.0, z=2.4)),
    attach_to=vehicle)
lidar_queue = queue.Queue(maxsize=2)
def _lidar_cb(data):
    if lidar_queue.full():
        try: lidar_queue.get_nowait()
        except queue.Empty: pass
    lidar_queue.put_nowait(data)
lidar_sensor.listen(_lidar_cb)

# — Датчик препятствий (obstacle) —
obs_bp = bp_lib.find('sensor.other.obstacle')
obs_bp.set_attribute('distance', '12')
obs_bp.set_attribute('hit_radius', '0.5')
obs_bp.set_attribute('only_dynamics', 'False')
obs_sensor = world.spawn_actor(
    obs_bp, carla.Transform(), attach_to=vehicle)
obstacle_flag = {'on': False}
def _obs_cb(event):
    if 'vehicle' in event.other_actor.type_id or \
        'walker' in event.other_actor.type_id:
        obstacle_flag['on'] = True
obs_sensor.listen(_obs_cb)

# — Датчик столкновения —
coll_bp = bp_lib.find('sensor.other.collision')
coll_sensor = world.spawn_actor(
    coll_bp, carla.Transform(), attach_to=vehicle)
collision_flag = {'hit': False}
def _coll_cb(event):
    collision_flag['hit'] = True
coll_sensor.listen(_coll_cb)

# — Видео —
vw = None
if RECORD:
    vw = cv2.VideoWriter(OUT_VIDEO, cv2.VideoWriter_fourcc(*'mp4v'),
        FPS, (640, 480))

# — Прорев —
print("❌ Прорев (60 тиков) ...", end="", flush=True)
for _ in range(60):
    world.tick()
print("👌 готово.\n")

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

print("\n Тест запущен. Q — выход, R — ручной respawn\n")

prev_steer = 0.0
prev_throt = 0.0
collisions = 0
respawns = 0
frame_cnt = 0
GRACE = 40 # тиков после спавна — не реагируем на препятствия
gru_h = None # скрытое состояние GRU; сбрасывается при respawn

try:
    while True:
        world.tick()

        # — Получить кадры —————
        try:
            cam_data = img_queue.get(timeout=0.5)
            lidar_data = lidar_queue.get(timeout=0.5)
        except queue.Empty:
            continue

        raw = np.frombuffer(cam_data.raw_data, dtype=np.uint8)
        img_bgr = cv2.cvtColor(
            raw.reshape(480, 640, 4)[: , : , :3], cv2.COLOR_RGB2BGR)
        pts = np.frombuffer(lidar_data.raw_data, dtype=np.float32).reshape(-1, 4)
        bev = lidar_to_bev(pts)

        # — Состояние машины —————
        vel = vehicle.get_velocity()
        speed = 3.6 * (vel.x**2 + vel.y**2 + vel.z**2) ** 0.5
        spd_norm = float(np.clip(speed / MAX_SPEED_KMH, 0.0, 1.0))
        cmd = get_command(vehicle, world_map)

        # — Столкновение → respawn —————
        if collision_flag['hit'] and RESPAWN_AFTER_COLLISION:
            collision_flag['hit'] = False
            collisions += 1
            tqdm_print = f" Столкновение, #{collisions} → respawn через 3 сек"
            print(tqdm_print)

            # Покажем кадр аварии
            cv2.putText(img_bgr, f"COLLISION #{collisions}",
                (150, 240), cv2.FONT_HERSHEY_SIMPLEX, 1.5,
                (0, 0, 255), 4)
            if vw: vw.write(img_bgr)
            cv2.imshow("CILDiveNet TEST", img_bgr)
            cv2.waitKey(1)

            # Пауза перед respawn
            for _ in range(RESPAWN_PAUSE_TICKS):
                world.tick()

            respawn_vehicle(vehicle, world, spawn_points,
                camera, lidar_sensor, img_queue, lidar_queue)
            respawns += 1
            frame_cnt = 0
            prev_steer = 0.0
            prev_throt = 0.0
            gru_h = None # сброс памяти после аварии
            continue

        # — Инференс модели —————
        with torch.no_grad():
            img_t = preprocess_image(img_bgr).to(device)
            bev_t = torch.from_numpy(bev).unsqueeze(0).to(device)
            spd_t = torch.tensor([spd_norm]).to(device)
            # GRU: передаём скрытое состояние между тиками
            actions, gru_h = model(img_t, bev_t, spd_t, h=gru_h)
            steer_raw, throt_raw = actions[0, cmd].cpu().numpy()

            steer_raw = float(steer_raw)
            throt_raw = float(throt_raw)

            # — Сглаживание —————
            steer = STEER_SM * prev_steer + (1 - STEER_SM) * steer_raw
            throt = THROT_SM * prev_throt + (1 - THROT_SM) * throt_raw

            # — Ограничение скорости —————
            if speed > TARGET_SPD:

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

        excess = (speed - TARGET_SPD) / TARGET_SPD
        throt = max(0.0, throt * (1.0 - excess))

    steer = float(np.clip(steer, -1.0, 1.0))
    throt = float(np.clip(throt, 0.0, 1.0))
    brake = 0.0

    # — Торможение перед перекрёстком —————
    if junction_ahead(vehicle, world_map, JUNCTION_DECEL_M):
        throt = min(throt, 0.30)
        if speed > JUNCTION_SPD:
            brake = 0.20

    # — Двухуровневая защита от столкновений —————
    in_grace = frame_cnt < GRACE
    obs_hit = obstacle_flag['on'] and not in_grace
    lidar_hit = obstacle_ahead(pts) and not in_grace
    obstacle_flag['on'] = False

    emerg = obs_hit or lidar_hit
    if emerg:
        throt = 0.0
        brake = 0.8

    # Минимальный газ при малой скорости (не даём встать)
    if speed < 2.0 and not emerg:
        throt = max(throt, 0.15)

    prev_steer = steer
    prev_throt = throt

    # — Применить управление —————
    vehicle.apply_control(carla.VehicleControl(
        throttle=throt, steer=steer, brake=brake,
        manual_gear_shift=False))

    # — HUD + дисплей —————
    draw_hud(img_bgr, steer, throt, speed, cmd, emerg, collisions, respawns)
    if vw:
        vw.write(img_bgr)
    cv2.imshow("CILDiveNet TEST", img_bgr)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'):
        print("\n[Q] Выход.")
        break
    elif key == ord('r'):
        print("[R] Ручной respawn")
        respawn_vehicle(vehicle, world, spawn_points,
                        camera, lidar_sensor, img_queue, lidar_queue)
        respawns += 1
        frame_cnt = 0
        prev_steer = prev_throt = 0.0
        gru_h = None

    frame_cnt += 1
    if frame_cnt % 200 == 0:
        print(f" [{frame_cnt:5d}] CMD={CMD_NAMES[cmd]:8s} "
              f"S={steer:.3f} T={throt:.3f} "
              f"V={speed:.1f} km/h "
              f"COLL={collisions} RESP={respawns}")

except KeyboardInterrupt:
    pass
finally:
    camera.stop(); camera.destroy()
    lidar_sensor.stop(); lidar_sensor.destroy()
    obs_sensor.stop(); obs_sensor.destroy()
    coll_sensor.stop(); coll_sensor.destroy()
    vehicle.destroy()
    for npc in npcs:
        try: npc.destroy()
        except Exception: pass
    tm.set_synchronous_mode(False)
    if vw:
        vw.release()
        print(f"[Q] Видео: {OUT_VIDEO}")
    cv2.destroyAllWindows()
    s = world.get_settings()

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

s.synchronous_mode = False
world.apply_settings(s)
print("🚦 Готово.")
print(f"Итого: столкновений={collisions} respawn-ов={respawns} кадров={frame cnt}")

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--ckpt', default='checkpoints/best_model.pth')
    parser.add_argument('--town', default='Town10HD')
    main(parser.parse_args())

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

ПРИЛОЖЕНИЕ И

Листинг модуля clean_dataset.py (очистка датасета)

```
"""
Полностью очищает датасет: удаляет все файлы в папках images/ и lidar/,
и стирает содержимое labels.csv (оставляет только заголовки).

Использование:
python scripts/clean_dataset.py
python scripts/clean_dataset.py --dry-run # только показать, что будет удалено
"""
import os
import glob
import argparse
import pandas as pd

DATA_DIR = os.path.join("data", "train")
CSV_PATH = os.path.join(DATA_DIR, "labels.csv")
IMAGES_DIR = os.path.join(DATA_DIR, "images")
LIDAR_DIR = os.path.join(DATA_DIR, "lidar")

def count_files(directory, pattern="*"):
    if not os.path.exists(directory):
        return 0
    return len(glob.glob(os.path.join(directory, pattern)))

def delete_all_files(directory):
    if not os.path.exists(directory):
        return 0
    files = glob.glob(os.path.join(directory, "*"))
    for f in files:
        if os.path.isfile(f):
            os.remove(f)
    return len(files)

def main(dry_run=False):
    img_count = count_files(IMAGES_DIR)
    lid_count = count_files(LIDAR_DIR)

    csv_rows = 0
    if os.path.exists(CSV_PATH):
        df = pd.read_csv(CSV_PATH)
        csv_rows = len(df)

    print(f"Найдено: {csv_rows} строк в CSV, {img_count} файлов в images/, {lid_count} файлов в lidar/")

    if dry_run:
        print("DRY RUN --- (файлы не удалены)")
        return

    deleted_img = delete_all_files(IMAGES_DIR)
    deleted_lid = delete_all_files(LIDAR_DIR)

    # Очищаем CSV - оставляем только заголовки
    if os.path.exists(CSV_PATH):
        df_empty = df.iloc[0:0]
        df_empty.to_csv(CSV_PATH, index=False)
        print(f"CSV очищен (заголовки сохранены)")
    else:
        print("labels.csv не найден, пропускаем")

    print(f"Удалено: {deleted_img} файлов из images/, {deleted_lid} файлов из lidar/")

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--dry-run', action='store_true',
                        help='Показать что будет удалено, не удалять')
    args = parser.parse_args()
    main(dry_run=args.dry_run)
```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

ПРИЛОЖЕНИЕ К

Листинг модуля plot_results.py (построение графиков обучения)

```
"""
Генерация графиков для дипломной работы.

Использование:
# После сбора данных:
python scripts/plot_results.py --mode dataset

# После обучения (читает TensorBoard логи):
python scripts/plot_results.py --mode training

# Всё сразу:
python scripts/plot_results.py --mode all

Все графики сохраняются в папку plots/
"""

import os
import sys
import argparse
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg') # без GUI — для сохранения в файл
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

PLOTS_DIR = "plots"
DATA_CSV = os.path.join("data", "train", "labels.csv")
TENSORBOARD = os.path.join("logs", "tensorboard")

CMD_NAMES = {0: "FOLLOW", 1: "LEFT", 2: "RIGHT", 3: "STRAIGHT"}
CMD_COLORS = {0: "#4C72B0", 1: "#DD8452", 2: "#55A868", 3: "#C44E52"}

os.makedirs(PLOTS_DIR, exist_ok=True)

# -----
# Helpers
# -----

def save(fig, name):
    path = os.path.join(PLOTS_DIR, name)
    fig.savefig(path, dpi=150, bbox_inches='tight')
    plt.close(fig)
    print(f" 📁 {path}")

# -----
# DATASET plots
# -----

def plot_dataset(csv_path=DATA_CSV):
    print(f" 📄 Генерация графиков датасета ...")
    df = pd.read_csv(csv_path)
    n = len(df)
    print(f"   Всего кадров: {n}")

    # — 1. Command distribution (pie + bar) —————
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    fig.suptitle("Распределение навигационных команд в датасете", fontsize=14, fontweight='bold')

    cmd_counts = df['command'].value_counts().sort_index()
    labels = [CMD_NAMES.get(i, str(i)) for i in cmd_counts.index]
    colors = [CMD_COLORS.get(i, '#999') for i in cmd_counts.index]

    axes[0].pie(cmd_counts.values, labels=labels, colors=colors,
                autopct='%1.1f%%', startangle=90,
                wedgeprops={'edgecolor': 'white', 'linewidth': 1.5})
    axes[0].set_title("Доля команд (pie chart)")

    axes[1].bar(labels, cmd_counts.values, color=colors, edgecolor='black', linewidth=0.7)
    axes[1].set_xlabel("Команда")
    axes[1].set_ylabel("Количество кадров")
```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

axes[1].set_title("Количество кадров по командам")
for i, v in enumerate(cmd_counts.values):
    axes[1].text(i, v + 20, f'{v}\n{100*v/n:.1f}% ', ha='center', fontsize=9)

plt.tight_layout()
save(fig, "01_command_distribution.png")

# — 2. Steering angle distribution —————
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle("Распределение угла руля (steer)", fontsize=14, fontweight='bold')

axes[0].hist(df['steer'], bins=80, color='steelblue', edgecolor='black',
             linewidth=0.4, alpha=0.85)
axes[0].axvline(0, color='red', linestyle='--', linewidth=1.2, label='центр')
axes[0].set_xlabel("Угол руля [-1, 1]")
axes[0].set_ylabel("Количество кадров")
axes[0].set_title("Полное распределение")
axes[0].legend()

# Per-command overlay
for cmd_id, name in CMD_NAMES.items():
    sub = df[df['command'] == cmd_id]['steer']
    if len(sub) > 0:
        axes[1].hist(sub, bins=60, alpha=0.6, label=name,
                     color=CMD_COLORS[cmd_id], edgecolor='none')
axes[1].axvline(0, color='black', linestyle='--', linewidth=1)
axes[1].set_xlabel("Угол руля [-1, 1]")
axes[1].set_ylabel("Количество кадров")
axes[1].set_title("По командам")
axes[1].legend()

plt.tight_layout()
save(fig, "02_steer_distribution.png")

# — 3. Throttle & speed distributions —————
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
fig.suptitle("Распределение газа и скорости", fontsize=14, fontweight='bold')

axes[0].hist(df['throttle'], bins=60, color='#55A868', edgecolor='black', linewidth=0.4)
axes[0].set_xlabel("Throttle [0, 1]")
axes[0].set_ylabel("Количество кадров")
axes[0].set_title("Газ (throttle)")

axes[1].hist(df['speed_kmh'], bins=60, color='#C44E52', edgecolor='black', linewidth=0.4)
axes[1].set_xlabel("Скорость (км/ч)")
axes[1].set_ylabel("Количество кадров")
axes[1].set_title("Скорость автомобиля")

plt.tight_layout()
save(fig, "03_throttle_speed_distribution.png")

# — 4. Steer timeline (первые 2000 кадров) —————
fig, axes = plt.subplots(3, 1, figsize=(14, 8), sharex=True)
fig.suptitle("Траектория управления (первые 2 000 кадров)", fontsize=14, fontweight='bold')

sample = df.head(2000)
axes[0].plot(sample.index, sample['steer'], color='steelblue', linewidth=0.8)
axes[0].axhline(0, color='red', linestyle='--', linewidth=0.8)
axes[0].set_ylabel("Руль (steer)")
axes[0].set_ylim(-1.1, 1.1)

axes[1].plot(sample.index, sample['throttle'], color='green', linewidth=0.8)
axes[1].set_ylabel("Газ (throttle)")
axes[1].set_ylim(-0.05, 1.05)

axes[2].plot(sample.index, sample['speed_kmh'], color='orange', linewidth=0.8)
axes[2].set_ylabel("Скорость (км/ч)")
axes[2].set_xlabel("Кадр")

plt.tight_layout()
save(fig, "04_control_timeline.png")

# — 5. Correlation: steer vs speed —————
fig, ax = plt.subplots(figsize=(8, 6))
sc = ax.scatter(df['speed_kmh'], df['steer'].abs(),
               c=df['command'], cmap='tab10', alpha=0.2, s=2,
               vmin=0, vmax=3)
from matplotlib.lines import Line2D
legend_els = [Line2D([0], [0], marker='o', color='w',
                    markerfacecolor=plt.cm.tab10(i / 10), markersize=8,

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

        label=CMD_NAMES[i] for i in range(4)]
ax.legend(handles=legend_els, title="Команда")
ax.set_xlabel("Скорость (км/ч)")
ax.set_ylabel("Руль (абс. значение)")
ax.set_title("Зависимость угла руля от скорости (по командам)")
plt.tight_layout()
save(fig, "05_steer_vs_speed.png")

print(f"✅ Графики датасета сохранены в {PLOTS_DIR}/")

# -----
# TRAINING plots (from TensorBoard event files)
# -----

def read_tb(logdir):
    """Read scalar summaries from TensorBoard event files."""
    try:
        from tensorboard.backend.event_processing.event_accumulator import EventAccumulator
    except ImportError:
        print("⚠️ 'tensorboard' не установлен для чтения логов напрямую.")
        print("❌ Занустить tensorboard --logdir logs/tensorboard")
        return None

    ea = EventAccumulator(logdir)
    ea.Reload()
    tags = ea.Tags().get('scalars', [])
    data = {}
    for tag in tags:
        events = ea.Scalars(tag)
        data[tag] = {
            'step': np.array([e.step for e in events]),
            'value': np.array([e.value for e in events]),
        }
    return data

def plot_training(logdir=TENSORBOARD):
    print("❌ Генерация графиков обучения ...")
    data = read_tb(logdir)
    if data is None:
        return

    # — 6. Train / Val Loss —————
    fig, axes = plt.subplots(1, 3, figsize=(16, 5))
    fig.suptitle("Кривые обучения CILDriveNet", fontsize=14, fontweight='bold')

    def plot_pair(ax, train_tag, val_tag, title, ylabel):
        if train_tag in data:
            ax.plot(data[train_tag]['step'], data[train_tag]['value'],
                    label='Train', color='steelblue', linewidth=1.5)
        if val_tag in data:
            ax.plot(data[val_tag]['step'], data[val_tag]['value'],
                    label='Val', color='#C44E52', linewidth=1.5)
        ax.set_title(title); ax.set_xlabel("Эпоха"); ax.set_ylabel(ylabel)
        ax.legend(); ax.grid(True, alpha=0.3)

    plot_pair(axes[0], 'Loss/train', 'Loss/val', 'Суммарный Loss', 'Loss')
    plot_pair(axes[1], 'Steer/train', 'Steer/val', 'Steer Loss', 'Loss')
    plot_pair(axes[2], 'Throttle/train', 'Throttle/val', 'Throttle Loss', 'Loss')

    plt.tight_layout()
    save(fig, "06_training_curves.png")

    # — 7. Per-command validation loss —————
    cmd_tags = ['Val/cmd_0', 'Val/cmd_1', 'Val/cmd_2', 'Val/cmd_3']
    available = [t for t in cmd_tags if t in data]
    if available:
        fig, ax = plt.subplots(figsize=(10, 5))
        fig.suptitle("Валидационный Loss по командам (branch losses)",
                     fontsize=14, fontweight='bold')
        for tag in available:
            cmd_id = int(tag.split('_')[-1])
            ax.plot(data[tag]['step'], data[tag]['value'],
                    label=CMD_NAMES[cmd_id], color=CMD_COLORS[cmd_id], linewidth=1.8)
        ax.set_xlabel("Эпоха"); ax.set_ylabel("Loss")
        ax.legend(); ax.grid(True, alpha=0.3)
        plt.tight_layout()
        save(fig, "07_per_command_loss.png")

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

print(f"📁 Графики обучения сохранены в {PLOTS_DIR}/")

# -----
# Architecture diagram (схема модели для диплома)
# -----

def plot_architecture():
    print("📄 Генерация схемы архитектуры ...")

    fig = plt.figure(figsize=(16, 8))
    ax = fig.add_subplot(111)
    ax.set_xlim(0, 16); ax.set_ylim(0, 8)
    ax.axis('off')
    fig.suptitle("Архитектура CILDriveNet (Conditional Imitation Learning)",
                fontsize=14, fontweight='bold')

    def box(ax, x, y, w, h, label, sub='', color='#AED6F1', fontsize=9):
        rect = plt.Rectangle((x, y), w, h, linewidth=1.5,
                             edgecolor='#2C3E50', facecolor=color, zorder=3)
        ax.add_patch(rect)
        ax.text(x + w/2, y + h/2 + (0.15 if sub else 0), label,
                ha='center', va='center', fontsize=fontsize, fontweight='bold', zorder=4)
        if sub:
            ax.text(x + w/2, y + h/2 - 0.25, sub,
                    ha='center', va='center', fontsize=7, color='#555', zorder=4)

    def arrow(ax, x1, y1, x2, y2):
        ax.annotate('', xy=(x2, y2), xytext=(x1, y1),
                    arrowprops=dict(arrowstyle='->', color='#2C3E50', lw=1.5), zorder=5)

    # Inputs
    box(ax, 0.2, 5.5, 2.0, 1.0, 'Camera', '3×224×224', '#AED6F1')
    box(ax, 0.2, 3.5, 2.0, 1.0, 'LIDAR BEV', '5×200×200', '#A9DFBF')
    box(ax, 0.2, 1.5, 2.0, 1.0, 'Speed', '1 (норм.)', '#F9E79F')

    # Encoders
    box(ax, 3.0, 5.5, 2.2, 1.0, 'ResNet-18', 'pretrained → 512d', '#AED6F1')
    box(ax, 3.0, 3.5, 2.2, 1.0, 'LidarCNN', '3×Conv → 128d', '#A9DFBF')
    box(ax, 3.0, 1.5, 2.2, 1.0, 'SpeedMLP', 'FC(1-64)', '#F9E79F')

    # Fusion
    box(ax, 6.2, 3.5, 2.0, 1.0, 'Fusion', 'FC(704-256)', '#E8DAEF')

    # Command
    box(ax, 6.0, 1.2, 2.2, 0.9, 'Command', '0-3 (капча)', '#FDEBD0')

    # Branches
    bcolors = ['#AED6F1', '#A9DFBF', '#F9E79F', '#FAD7A0']
    bnames = ['Branch 0\nFOLLOW', 'Branch 1\nLEFT',
              'Branch 2\nRIGHT', 'Branch 3\nSTRAIGHT']
    for i, (name, col) in enumerate(zip(bnames, bcolors)):
        yb = 6.5 - i * 1.5
        box(ax, 9.5, yb, 2.2, 1.0, name, 'FC(256-128-2)', col)
        arrow(ax, 8.2, 4.0, 9.5, yb + 0.5)

    # Output
    box(ax, 12.5, 3.5, 2.5, 1.0, 'Output', '[steer, throttle]', '#FADBD8', fontsize=10)
    for i in range(4):
        yb = 6.5 - i * 1.5 + 0.5
        arrow(ax, 11.7, yb, 12.5, 4.0)

    # Arrows encoders → fusion
    arrow(ax, 2.2, 6.0, 3.0, 6.0)
    arrow(ax, 2.2, 4.0, 3.0, 4.0)
    arrow(ax, 2.2, 2.0, 3.0, 2.0)
    arrow(ax, 5.2, 6.0, 6.2, 4.3)
    arrow(ax, 5.2, 4.0, 6.2, 4.0)
    arrow(ax, 5.2, 2.0, 6.2, 3.7)

    # Command → selector hint
    ax.text(8.2, 1.65, '← активная ветвь', fontsize=8, color='#8B0000')

    plt.tight_layout()
    save(fig, "00_architecture.png")
    print(f"Схема архитектуры сохранена в {PLOTS_DIR}/")

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

```

# Entry point
def main(args):
    if args.mode in ('dataset', 'all'):
        if os.path.exists(DATA_CSV):
            plot_dataset(DATA_CSV)
        else:
            print(f" Не найден {DATA_CSV} – сначала соберите данные.")

    if args.mode in ('training', 'all'):
        if os.path.isdir(TENSORBOARD):
            plot_training(TENSORBOARD)
        else:
            print(f" Не найден {TENSORBOARD} – сначала запустите обучение.")

    if args.mode in ('arch', 'all'):
        plot_architecture()

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description="Графики для дипломной работы")
    parser.add_argument('--mode', choices=['dataset', 'training', 'arch', 'all'],
                        default='all', help="Что генерировать")
    main(parser.parse_args())

```

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)

отформатировано: английский (США)